

UNIVERSITY OF TRIESTE

Department of Mathematics, Informatics and
Geosciences



Master's Degree in Data Science and Artificial
Intelligence
Curriculum: Foundations of Artificial Intelligence and Machine
Learning

Unsupervised State Bucketing for Mixture of Experts in Resource-Constrained Chess Engines

Graduation session: 23 October 2026

Candidate
Omar Cusma Fait
Student ID SM3800018

Supervisor
Prof. Alessio Ansuini

Internship supervisor
Prof. Marco Zennaro
ICTP

Academic Year 2025/2026

Divide and conquer.

Abstract

In the context of board games, a common approach to increasing model performance is to partition the state space and allocate distinct experts to each region. While this is the central idea behind Mixture of Experts (MoE) architectures, the bucketing is almost always defined manually, requiring domain-specific expertise, and yielding potentially suboptimal partitions.

While unsupervised bucketing using hidden layer activations has been explored, this approach may not yield partitions that *maximise expert specialisation*. Activations capture what the model *knows*, not what it *needs* to adjust. In this work, we ask whether we can use the model’s learning dynamics, the sample-gradients, to create a partition that is more effective. Our hypothesis is that clustering gradients (which encode the direction of desired weight updates) will produce buckets that are inherently better suited for expert fine-tuning, leading to more specialised experts.

We propose an unsupervised bucketing method based on sample-gradients: the gradients of the loss with respect to the head parameters of a base model. These vectors encode the direction in weight space that would improve the model’s prediction for each individual data point. We apply the method to chess, using a standard NNUE (Efficiently Updatable Neural Network) as the base model trained on 105 million positions labelled with outcome probabilities (WDL) by a strong value function (Lc0).

For each position, we compute the normalised sample-gradient with respect to the head parameters, and apply a clustering algorithm to define the buckets. We then train a lightweight linear dispatcher to predict the bucket from the L1 activations, enabling fast, deterministic routing at inference time. Finally, we fine-tune the expert heads on each bucket, starting from the base model, while keeping the L1 weights frozen. This yields a set of fast, specialised experts, each adapted to a distinct region of the state space, without requiring handcrafted bucketing.

Contents

Abstract	i
1 Introduction	1
1.1 Motivation	1
1.2 Problem Statement	2
1.3 Contributions	2
1.4 Thesis Outline	3
2 Background and Related Work	5
2.1 Chess Engines and Evaluation Functions	5
2.1.1 Classical Evaluation	5
2.1.2 The Shift to Neural Evaluation	6
2.1.3 NNUE: A Hybrid Approach	7
2.1.4 Value Targets: Centipawns, Expected Value, and WDL	7
2.1.5 Why Evaluation Must Be Cheap	9
2.1.6 Current State of the Art for Tiny Hardware: Cfish . .	9
2.2 Mixture of Experts	10
2.2.1 Fixed vs. Learned Routing	10
2.2.2 Applications in Vision, NLP, and Reinforcement Learning	11
2.2.3 LoRA as a Lightweight Expert Implementation	11
2.2.4 What Carries Over to a Tiny Chess Evaluation	12
2.3 Sample Gradients	12
2.3.1 Per-Sample Gradients	12
2.3.2 Gradient Similarity and Specialization	13
2.3.3 Gradient-Based Clustering	15
2.4 Bucketing in NNUE	16
2.4.1 Handcrafted Bucketing	16
2.4.2 Learned Bucketing	17
2.5 Related Work	19
2.5.1 Efficient Chess Engines	19
2.5.2 Teacher-Student Evaluation	20
2.5.3 Gradient-Based Expert Specialization	21

CONTENTS

3	Method	24
3.1	Method Overview	24
3.2	Notation and Definitions	25
3.2.1	Sets	25
3.2.2	Scalars	25
3.2.3	Vectors and Matrices	26
3.2.4	Functions and Models	26
3.2.5	Key Operators	26
3.2.6	Relationship Between Δ_i and δ_i	27
3.3	Step 1: Train the Base Model	27
3.3.1	Model Architecture	27
3.3.2	Training Objective	27
3.3.3	Training Protocol	28
3.3.4	The Role of the Base Model	28
3.3.5	Why Freeze L1?	28
3.4	Step 2: Compute Sample Gradients	29
3.4.1	Definition of Sample Gradient	29
3.4.2	Implementation	29
3.4.3	Normalisation	29
3.4.4	Storage and Compute Considerations	30
3.4.5	Handling Duplicate Positions	30
3.5	Step 3: Cluster Sample Gradients	31
3.5.1	The Objective of Clustering	31
3.5.2	Fixed- B Algorithms: K-Means and Variants	31
3.5.3	Density-Based Algorithms	32
3.5.4	Choosing B and the Algorithm	32
3.5.5	Validation and Diagnostics	32
3.6	Step 4: Train the Dispatcher	34
3.6.1	Architecture	34
3.6.2	Why a Linear Model	34
3.6.3	Training	35
3.6.4	Can a Linear Dispatcher Capture the Clustering?	35
3.6.5	Fixed vs. Re-Assigned Cluster Assignments	35
3.7	Step 5: Fine-Tune Expert Heads	36
3.7.1	What Is Trained and What Is Frozen	36
3.7.2	Training Procedure	36
3.7.3	Data Availability per Expert	37
3.7.4	The Resulting MoE Model	37
3.8	Generalisation Beyond Chess	38
4	Implementation	40
4.1	Dataset and Teacher	40
4.1.1	Dataset Construction	40
4.1.2	Feature Encoding	41

CONTENTS

4.1.3	Data Splits	41
4.1.4	Teacher Model: Lc0	41
4.1.5	Labelling Pipeline	42
4.2	Base NNUE Architecture	42
4.2.1	Model Overview	42
4.2.2	Shared L1 Accumulator	44
4.2.3	Side-to-Move Reorder	44
4.2.4	L2 Layer and Output Head	44
4.2.5	Training Objective	45
4.2.6	Parameter Count and Model Size	45
4.2.7	Training Protocol	46
4.3	Sample Gradient Computation	46
4.3.1	Implementation with PyTorch	46
4.3.2	Parameter Selection	47
4.3.3	Normalisation	47
4.3.4	Storage and Memory Management	48
4.3.5	Computational Cost and Timing	48
4.4	Clustering	49
4.5	Dispatcher Training	50
4.6	Expert Fine-Tuning	51
4.7	Final MoE Model	52
4.8	Cfish as the Host Engine	53
4.9	Integration with Cfish	54
5	Experiments and Results	56
5.1	Experimental Setup	56
5.2	Evaluation Metrics	56
5.3	Results	56
5.3.1	Linear Model	56
5.3.2	Single-layer FFNN	56
5.3.3	Double-layer FFNN	56
5.3.4	Base NNUE	56
5.3.5	MoE NNUE — bucketing by L1 — fixed B	57
5.3.6	MoE NNUE — bucketing by L1 — variable B	57
5.3.7	MoE NNUE — bucketing with sample-gradients — fixed B	57
5.3.8	MoE NNUE — bucketing with sample-gradients — variable B	57
5.4	Results: Comparison of the Models	57
5.5	Visualization: Clustering	57

CONTENTS

6	Discussion	59
6.1	Interpretation of Results	59
6.2	Limitations	59
6.3	Generalisation	59
6.4	Comparison with Related Work	59
7	Conclusion and Future Work	60
7.1	Summary of Contributions	60
7.2	Future Work	60
7.3	Closing Remarks	60
A	Supplementary Material	61
A.1	Extra tables and hyperparameters	61
A.2	Extra plots	61
A.3	Implementation notes	61

Chapter 1

Introduction

1.1 Motivation

A chess engine evaluates a large number of positions during search, and the quality of this evaluation directly affects the quality of the resulting moves. On a desktop system, this evaluation can rely on relatively large neural networks. On a microcontroller, memory, computation, and latency constraints impose much tighter limits on the size and cost of the evaluation function. In the latter case, the evaluation function must be small, integer-friendly, and cheap to run at every node of an alpha-beta search. *Efficiently Updatable Neural Networks* (NNUE) architectures have become a widely used approach for neural evaluation in modern chess engines.

A single shallow head must represent positions arising from substantially different tactical and positional regimes. This raises the question of whether different regions of the state space could benefit from specialized heads. Different positions may require understanding of very diverse aspects of the game. Mixture-of-Experts (MoE) evaluation provides a natural framework for this form of specialization: partition the state space into buckets, and assign an *expert head* to each region, while keeping a shared representation. Common chess-engine bucketing strategies rely on manually designed features such as piece count, king location, or game phase. Those rules are cheap and interpretable, but they are game-specific heuristics and are not necessarily optimal. They encode what a programmer thinks is a distinct regime, not what the model actually needs in order to specialise.

Unsupervised alternatives exist. Clustering hidden-layer activations, for example, groups positions that look similar in the eyes of the network. The weakness of this approach lies in the fact that **similarity of representation is not the same as similarity of learning signal**. Hidden activations characterize the representation produced by the current model, whereas gradients with respect to the head parameters directly characterize how the loss would change under parameter updates. This motivates defining the

partition in terms of the parameter-space directions induced by individual training samples. This thesis is motivated by the idea that **lightweight, data-driven bucketing can provide a practical solution for on-device NNUE evaluation** while **avoiding reliance on chess-specific features**. More generally, such an approach provides a way to mix experts over a state space using information derived from the learning process.

1.2 Problem Statement

The central problem addressed in this thesis is the design of an efficient, *data-driven* method for partitioning the state space of a chess engine into regions of specialist expertise, within the tight *inference-time* resource constraints of embedded devices.

More specifically, consider a standard NNUE evaluation function composed of a frozen shared representation W_{L1} and a trainable head (W_{L2}, W_{out}) . Given a dataset of positions $\mathcal{D} = \{(s_i, v_i)\}$, we can train a base model w_{base} . To improve upon this base model via expert specialization, we seek an algorithm that can effectively partition the state space into B buckets $\{\mathcal{D}_1, \dots, \mathcal{D}_B\}$ such that fine-tuning a separate head on each bucket yields a set of specialized models with *distinct parameter updates*. We refer to the parameter difference $\delta_i = \theta_i - \theta_{\text{base}}$ as a *task vector*, following the terminology commonly used for parameter-space representations of model specialization.

This objective is complicated by the fact that the partition must be discovered from the training data, yet the criterion for effective specialization is expressed in parameter space through the diversity of task vectors, rather than directly in the input space. At the same time, the resulting routing mechanism must be *lightweight* enough to run on a microcontroller at every node of an alpha-beta search, ruling out expensive computations such as evaluating multiple full networks. Current heuristic bucketing strategies (e.g., based on piece count or king position) are computationally cheap but encode arbitrary *game-specific* assumptions about which positions are similar, which may not align with the learning signal relevant to head specialization.

This work, therefore, investigates whether it is possible to learn an unsupervised partition directly from instance-specific gradients, using them as a proxy for the direction in which each head should specialize, and whether such a partition can be deployed via a lightweight dispatcher that respects the memory and computational constraints of an embedded device, without sacrificing the quality of the resulting mixture-of-experts evaluation.

1.3 Contributions

Recent work has explored the use of gradient directions to identify groups of samples with related optimization behavior and to construct specialized

models. **ELREA** [1], for example, partitions training instructions according to their gradient directions to reduce optimization conflicts, while **GradientSpace** [2] clusters LoRA gradients and uses a lightweight encoder-based router to enable single-expert inference. These works provide evidence that gradient-space structure can be exploited to identify forms of specialization that are not directly defined by the input space.

However, these approaches target large language models and operate under assumptions that differ substantially from those of embedded chess engines. In our setting, the evaluation function may be invoked at every node of an alpha-beta search, making both the computational cost of routing and the memory footprint of the experts critical constraints. Furthermore, the desired partition should be learned from the training data without relying on manually designed chess-specific features.

Building on this perspective, we investigate a gradient-informed approach to state-space partitioning for NNUE evaluation. We propose a Mixture-of-Experts architecture in which positions are grouped by clustering their per-sample gradients with respect to the trainable head parameters, (W_{L2}, W_{out}) . The resulting clusters define candidate regions for expert specialization, while the shared L1 representation is kept common across all experts. At inference time, we introduce a lightweight dispatcher that predicts the bucket assignment from the frozen L1 activations and routes each position to a single specialized head. This separates the computationally expensive discovery of the partition from the lightweight routing required during search.

We evaluate the proposed approach on chess positions labeled with WDL values from a strong engine. The evaluation compares gradient-informed partitioning with single-head and heuristic bucketing baselines, examining both the specialization of the resulting experts and the quality and computational cost of the resulting evaluation function.

1.4 Thesis Outline

The next chapter introduces the background and related work underlying the proposed approach. It reviews chess evaluation functions, with particular attention to NNUE architectures, value representations, and the computational constraints of embedded chess engines. It then introduces Mixture-of-Experts architectures, per-sample gradients, and existing approaches to state-space bucketing and gradient-based expert specialization. The discussion concludes by positioning the proposed method with respect to existing work in efficient chess evaluation, teacher-student training, and gradient-based expert specialization.

The following chapter presents the proposed method for unsupervised state-space bucketing via sample gradients. It describes the training of the base NNUE, the computation and clustering of normalized per-sample gra-

dients, the construction of the buckets, and the training of a lightweight dispatcher for inference-time routing. The subsequent chapter describes the experimental setup and implementation, including the dataset and teacher model, network architecture, training procedure, quantization, and integration with the Cfish engine on the target embedded hardware. The experimental results are then presented and discussed, with particular attention to expert specialization, evaluation quality, computational cost, and comparison with the selected baselines. The thesis concludes by summarizing the main findings, discussing the limitations of the approach, and outlining possible directions for future work.

Chapter 2

Background and Related Work

2.1 Chess Engines and Evaluation Functions

At the core of every chess engine lies an **evaluation function** $f : \mathcal{S} \rightarrow \mathbb{R}$ that assigns a scalar value to a board position, indicating the expected outcome from that position (typically from the perspective of the player to move). This value guides the search algorithm, usually a variant of **alpha-beta search** with iterative deepening, by pruning unpromising branches and selecting the most promising moves.

2.1.1 Classical Evaluation

For decades, chess evaluation functions were handcrafted. A classical evaluator is a weighted linear combination of chess-specific features:

$$f(s) = \sum_k w_k \cdot \phi_k(s)$$

where $\phi_k(s)$ are feature functions and w_k are scalar weights. Typical chess-specific features include material balance, piece-square tables (PSTs), mobility, king safety, and pawn structure. Chess experts carefully tuned these features over decades, using a combination of intuition, empirical testing, and later automated optimization techniques.

A notable example is **PeSTO** (Piece-Square Tables Only), an evaluation function by Ronald Friederich that relies exclusively on piece-square tables. Its tables are optimized via *Texel*'s tuning method and use a *ta-pered evaluation* that interpolates between separate opening and endgame tables based on the game stage. Conceptually, PeSTO is equivalent to a linear feed-forward network: both compute a weighted sum of piece-square features, with no interaction terms between pieces. While still a handcrafted linear form, PeSTO demonstrates how far such classical approaches can be pushed through data-driven tuning.

While classical evaluators are extremely fast, they suffer from fundamental limitations. Human-designed features encode human intuition about chess, which may not align with the optimal understanding of the game. Moreover, these models do not take into account the positions as a whole; the same parameters give a positional bonus for a central knight whether the position is a quiet middlegame or a tactical melee, even though the knight’s practical value may differ dramatically across phases.

Classical evaluators are also vulnerable to the *horizon effect*. A search algorithm constrained to a fixed depth evaluates positions at the search frontier as if the game were stable at that point. If a tactical sequence begins at the horizon but extends one ply further, the engine cannot see the consequences and may assign an inaccurate evaluation. Classical evaluators, which rely on static features such as material balance and piece-square tables, are particularly susceptible: they treat a position at the horizon as if it were quiet, ignoring the tactical possibilities that would unfold if the search continued. In contrast, models trained on complete game outcomes can often encode patterns that extend beyond immediate material and positional features, making them less vulnerable to horizon-induced misevaluations.

2.1.2 The Shift to Neural Evaluation

The limitations of handcrafted features led to the adoption of neural networks as evaluation functions. **AlphaZero** [3] demonstrated that a deep convolutional network, trained via self-play reinforcement learning, could surpass the best classical engines. However, these networks are computationally expensive, requiring millions of operations per evaluation, making them unsuitable for resource-constrained devices or for engines that must evaluate millions of positions per second.

AlphaZero’s value head outputs a scalar $v \in [-1, +1]$, interpreted as the expected game outcome from the current player’s perspective. This scalar is the training target for the value network. However, AlphaZero’s training and inference rely on **Monte Carlo Tree Search (MCTS)**, which builds a search tree by repeatedly simulating trajectories and using the neural network to evaluate leaf nodes. This process requires many forward passes of the network per position, making it computationally expensive and poorly suited for engines that evaluate millions of positions per second with alpha-beta search. The AlphaZero network itself is a deep residual architecture with millions of parameters, requiring floating-point operations and substantial memory, which far exceeds the capacity of microcontrollers. In contrast, the NNUE architecture adopted in this work reduces the per-evaluation cost to a handful of integer operations while retaining the representational power of a neural network. We revisit AlphaZero’s value formulation in Section 2.1.4, where we contrast its scalar expected value with the WDL distribution used in this work.

2.1.3 NNUE: A Hybrid Approach

The **Efficiently Updatable Neural Network (NNUE)** architecture, first introduced in the *shogi* engine *YaneuraOu* and later adopted by *Stockfish*, strikes a pragmatic balance. NNUE combines the representational power of a neural network with the incremental efficiency of classical evaluators.

The architecture consists of two ingredients: a sparse first layer called the *accumulator*, and a small fully-connected head.

The accumulator layer maps a binary representation of the board to a hidden representation $h \in \mathbb{R}^d$ via $h = W_{L1} \cdot x$, where x is a sparse binary vector (typically 768 or 1024 dimensions) indicating the presence of pieces on squares. The key insight is that a chess move changes only a few bits of x , so h can be **updated incrementally** by adding and subtracting the corresponding columns of W_{L1} , rather than by recomputing the entire matrix-vector product from scratch. This makes the computation of L1 essentially *free*.

The second ingredient, the fully connected head, is typically one or two hidden layers followed by a scalar output, mapping h to the position value. In our case, we found it easier to train a probability distribution across all three possible game results for the player: win, draw, and loss (WDL), by minimizing the cross-entropy between the output of the teacher and the student.

A complete NNUE engine also leverages alpha-beta search with iterative deepening: starting from the root position, the engine searches deeper and deeper, using the NNUE evaluation at leaf nodes to guide the pruning and ordering of moves. At every node of this search tree, the evaluation function is called hundreds or thousands of times, making its speed critical. Also, being trained on enormous amounts of data, a NNUE doesn't suffer from the *horizon problem* as much as a PST does.

2.1.4 Value Targets: Centipawns, Expected Value, and WDL

A fundamental design choice in any chess evaluation function is the representation of the target value. Different engines adopt different representations, each with implications for training, calibration, and search integration. Three main approaches have emerged in practice: scalar centipawns, scalar expected value, and full WDL distributions.

Scalar centipawns

Classical evaluators and Stockfish-style NNUEs output a scalar value in centipawns, representing the expected advantage from the current player's perspective. A positive value indicates an advantage for the side to move; a negative value indicates a disadvantage. This representation is simple, interpretable, and compatible with existing search algorithms. However, it compresses the uncertainty of the game outcome into a single number: a

position with a 100 centipawn advantage might be a forced win, a quiet positional advantage, or a tactical trap, all of which require different handling during search. Moreover, training a network to predict centipawns typically uses mean squared error, which treats all deviations equally regardless of whether they lie near the decision boundary.

Scalar expected value (EV)

AlphaZero [3] adopted a different scalar representation: $v \in [-1, +1]$, interpreted as the expected game outcome from the current player’s perspective. This value is the difference between the probability of a win and the probability of a loss: $v = p_W - p_L$. A value of +1 indicates a certain win, −1 a certain loss, and 0 a draw or perfectly balanced position. This representation is more naturally calibrated to game outcomes than centipawns and avoids the arbitrary scaling of classical evaluation. However, like centipawns, it compresses the full distribution into a single scalar, losing information about the probability of a draw. A position with $v = 0$ could be a balanced middlegame, a drawish endgame, or a position where the engine is equally uncertain about win and loss, all of which have different implications for search.

Full WDL distribution

The approach adopted in this work is to train the NNUE to output a full probability distribution over the three possible game outcomes: Win, Draw, and Loss. The output head produces three logits, converted to probabilities via softmax:

$$p_{\text{WDL}}(s) = (p_W, p_D, p_L), \quad p_W + p_D + p_L = 1.$$

The scalar expected value used during search is then derived as $v = p_W - p_L$, but the network is trained to match the full distribution rather than just the scalar. This representation has several advantages. First, it preserves information about uncertainty: a position with $p_W = 0$, $p_D = 1$, $p_L = 0$ has the same scalar value as one with $p_W = 0.5$, $p_D = 0$, $p_L = 0.5$, but the two positions are fundamentally different. Second, it enables the use of **soft cross-entropy** as the loss function, which provides a richer training signal than mean squared error: the network is encouraged to match the full shape of the distribution, not just its mean. Third, the loss does not saturate at extreme values, as squared error on v would. Finally, the softmax output is naturally calibrated as a probability distribution, which can be useful for downstream tasks such as move selection or search heuristics.

Choice of loss and its implications

The choice of target representation determines the loss function. For scalar targets (centipawns or EV), mean squared error is the natural choice. For

WDL distributions, soft cross-entropy is more appropriate. In this work, we adopt the WDL representation with soft cross-entropy loss, as it provides the richest training signal while still yielding a scalar value compatible with alpha-beta search. This choice is reflected in the architecture (three output neurons instead of one) and in the teacher model (Lc0 natively outputs WDL probabilities). We revisit the practical implications of this choice in Chapter 4, where we describe the training procedure in detail.

2.1.5 Why Evaluation Must Be Cheap

The search tree explored by an **alpha-beta engine** grows exponentially with depth. Even with effective move ordering and pruning techniques, the number of evaluated positions increases dramatically as the search goes deeper. A chess engine that searches deeper consistently outperforms one that searches shallower, as each additional ply reveals tactical patterns and strategic nuances that would otherwise remain hidden.

This means that any overhead added to the evaluation function is directly subtracted from the engine’s effective search depth. If the evaluator becomes slower, the engine must reduce its search depth to maintain the same response time, sacrificing playing strength in the process.

On an embedded device such as the Wio Terminal (192 KB RAM, 512 KB flash), this constraint is even more severe. Memory is limited, floating-point operations are expensive, and every instruction counts. Any routing mechanism for a mixture-of-experts NNUE must add only a trivial cost, ideally a handful of integer operations or a simple table look-up, to avoid degrading the engine’s search performance.

In short, the evaluation function must be expressive enough to assess positions accurately, yet cheap enough to be called millions of times during a game. This trade-off is the central engineering challenge addressed by this thesis.

2.1.6 Current State of the Art for Tiny Hardware: Cfish

For resource-constrained devices, the most relevant reference implementation is **Cfish**, a port of the Stockfish chess engine written in plain C by Ronald de Man. While Stockfish is written in C++ and targets desktop-class hardware, Cfish strips away the C++ abstractions and compiles to a much smaller binary, making it viable for microcontrollers and other embedded platforms. The engine can be compiled with various evaluation backends, including a pure NNUE mode that excludes the classical handcrafted evaluation entirely, resulting in an even smaller executable.

From an architectural standpoint, Cfish shares the same core search and evaluation logic as Stockfish. The input to the evaluation function is a board position represented internally as a compact bitboard structure. The NNUE

evaluation itself follows the *halfkp* feature set: for each piece on the board, the network considers the piece’s square together with the square of the king of the same color, producing a set of activated features that are fed into the accumulator. The accumulator is updated incrementally as moves are made, avoiding recomputation from scratch. The output of the NNUE is a scalar value, typically in centipawns, representing the expected advantage from the perspective of the side to move.

Cfish is particularly relevant to this thesis for two reasons. First, it demonstrates that a full-featured NNUE engine *can* be made to run on constrained hardware, provided the implementation is careful about memory layout and avoids C++ overhead. Second, it serves as a practical baseline: any Mixture-of-Experts NNUE architecture proposed for tiny devices should be at least as fast and compact as Cfish in its pure NNUE mode, while offering improved evaluation accuracy through expert specialization. In this sense, Cfish represents both the *state of the art* and the *performance target* for the embedded chess engine developed in this work.

2.2 Mixture of Experts

The **Mixture of Experts (MoE)** architecture is a neural network design pattern in which multiple specialized sub-networks, or *experts*, are combined through a routing mechanism that selects or weights their contributions based on the input. The central idea is that different regions of the input space may require different processing, and dedicating separate capacity to each region can improve overall performance without substantially increasing the cost of a single forward pass.

2.2.1 Fixed vs. Learned Routing

MoE architectures can be broadly divided into two categories based on how the routing is determined: *fixed routing* and *learned routing*.

Fixed routing

In *fixed routing* schemes, the assignment of inputs to experts is determined by a predefined rule, often based on domain knowledge. In chess engines, this corresponds to handcrafted bucketing: positions are assigned to buckets based on material count, piece presence, or game phase. The routing is deterministic, interpretable, and computationally cheap, but it relies on human intuition about which regions of the state space are meaningfully distinct. The rule is fixed after design and cannot adapt to the data.

Learned routing

In *learned routing* schemes, a trainable *gating network* (or router) learns to assign inputs to experts based on the input features themselves. The gating network typically produces a probability distribution over experts, and the final output is a weighted combination of expert outputs, or a single expert selected by argmax. This approach is more flexible: the router can learn to assign inputs to experts in ways that may not align with human intuition, potentially discovering structure in the data that handcrafted rules would miss. However, learned routing introduces additional parameters and computational cost, and it may require careful design to avoid load imbalance or mode collapse.

2.2.2 Applications in Vision, NLP, and Reinforcement Learning

Mixture of Experts has been successfully applied across a wide range of domains. In natural language processing, large-scale MoE models such as the Switch Transformer [4] and GLaM [5] achieve strong performance by scaling the number of parameters while keeping per-token computation constant: only a subset of experts is activated for each token. In computer vision, MoE architectures have been used to efficiently scale convolutional networks and vision transformers, where different experts specialise in different visual patterns or object classes. In reinforcement learning, MoE has been applied to multi-task and multi-domain settings, where different experts specialise in different tasks or environments, and a gating network selects the appropriate expert for the current context.

Despite their success in these domains, MoE architectures are rarely deployed in resource-constrained settings such as microcontrollers. The gating network and the multiple expert heads introduce memory and computational overhead that is acceptable on servers but prohibitive on embedded devices. Moreover, many MoE implementations rely on sparse activation (only a subset of experts is used per forward pass) and require specialised hardware support for efficient execution. These constraints are less severe in the chess domain, where the evaluation function must be simple and fast, but they still shape the design choices of this work.

2.2.3 LoRA as a Lightweight Expert Implementation

In the context of large language models, a common approach to implementing experts is **Low-Rank Adaptation (LoRA)** [6]. LoRA freezes the base model’s weights and injects trainable low-rank matrices into each layer, enabling efficient fine-tuning with a small number of additional parameters. Each expert can be represented by a set of LoRA adapters that modify the base model’s behaviour in a task-specific or domain-specific way.

ELREA [1] and **GradientSpace** [2] both adopt this paradigm. In ELREA, training instructions are partitioned by their gradient directions, and a LoRA expert is fine-tuned on each partition. In GradientSpace, LoRA gradients are clustered, and a lightweight encoder-based router selects the appropriate LoRA expert for each input. These approaches demonstrate that gradient-informed partitioning can be effective, and they provide the closest methodological precedent for the work presented in this thesis.

However, LoRA is designed for large transformer models where the base model has hundreds of millions or billions of parameters. In our setting, the base model is a tiny NNUE with approximately 175,000 parameters, and the head that we specialise is already small. LoRA is therefore neither necessary nor appropriate: the expert heads are implemented as separate instances of the L2 and output layers, initialised from the base head and fine-tuned on their assigned buckets. This is simpler, more memory-efficient, and better suited to the integer quantisation required for deployment on microcontrollers.

2.2.4 What Carries Over to a Tiny Chess Evaluation

From the broader MoE literature, the key ideas that inform this work are the principle of partitioning the input space to enable specialisation, the distinction between fixed and learned routing, and the observation that gradient-based clustering can be used to discover meaningful partitions. However, the specific constraints of embedded chess engines impose a different set of trade-offs. The routing mechanism must be nearly free, ruling out expensive gating networks; the expert heads must be small and integer-friendly, ruling out LoRA adapters; and the partition must be learned from value estimation targets rather than instruction-following data. This thesis adapts the **gradient-based partitioning** paradigm to these constraints, proposing a lightweight linear dispatcher that routes positions to specialised heads with negligible overhead, and validating the approach on a chess NNUE for resource-constrained devices.

2.3 Sample Gradients

2.3.1 Per-Sample Gradients

In standard neural network training, the loss is computed over a mini-batch, and the resulting gradient is an average over the examples in that batch. This averaging discards information about how individual examples differ in their contribution to the update, treating the batch as a single unit. Per-sample gradient computation, by contrast, produces a separate gradient vector for each example in the batch, capturing the direction in which the model parameters would need to move to reduce the loss on that specific example

alone.

This quantity is not merely a computational curiosity. Per-sample gradients have found applications in *differential privacy*, where individual example contributions must be bounded to protect privacy, and in meta-learning, where the gradient with respect to a specific example informs task adaptation. More recently, a line of work has used per-sample gradients as a representational tool: the gradient of a single example encodes what that example wants the model to learn, and clustering these gradients reveals groups of examples that share similar learning requirements.

A key property of per-sample gradients is that they are directly defined in parameter space. The gradient

$$\Delta_i = \nabla_w \mathcal{L}(f_w(s_i), v_i)$$

specifies a direction in the same space in which the model’s weights reside. This distinguishes them from hidden-layer activations, which characterize the representation that the model currently produces for an input but say nothing about how that representation should change. Two examples may produce very similar activations because the model already evaluates them well, yet require different updates to correct subtle errors; conversely, examples with dissimilar activations may benefit from similar parameter adjustments. The gradient, by construction, resolves this distinction: it is the direction in which the loss would decrease most rapidly for that example, and therefore a direct measure of the learning signal.

The connection between per-sample gradients and expert specialization is supported by both theoretical and empirical work. Kawata et al. [7] prove that a Mixture of Experts trained with stochastic gradient descent can provably detect and exploit latent cluster structure in the data, dividing the problem into easier subproblems and achieving sample complexity gains over a single model. This result provides formal justification for using gradient information to guide the partitioning of a state space. Empirically, ELREA [1] and GradientSpace [2] demonstrate that clustering gradients, whether of full parameters or low-rank adapters, yields partitions that support effective expert specialization in language models. TAGCOS [8] extends the idea to coreset selection, using gradient clustering to identify representative examples for efficient fine-tuning. In the context of this thesis, the per-sample gradient is the bridge between the state space (which the dispatcher can observe at inference time) and the parameter space (where specialization actually occurs). It is the signal that determines which positions should be grouped together, and the remainder of this chapter builds on that principle.

2.3.2 Gradient Similarity and Specialization

If per-sample gradients encode what each example wants the model to learn, the natural next question is what it means for two gradients to be similar,

and why similarity should matter for specialization. The answer lies in the geometry of the gradient space and in the optimization dynamics it induces.

Two per-sample gradients are similar when they point in approximately the same direction in parameter space, that is, when their cosine similarity is high. This means that the parameter updates that would reduce the loss on one example would also reduce the loss on the other. When many examples share this property, they can be served by the same parameter configuration without conflict: a single update direction benefits all of them simultaneously. When gradients point in opposing directions, the update demanded by one example harms the other, and a single model must compromise, settling on parameters that are suboptimal for both.

This observation is the conceptual core of the gradient-clustering approaches discussed in the previous section. Clustering per-sample gradients groups together examples whose learning requirements are aligned, so that each cluster can be served by a dedicated expert without internal conflict. The partition is not defined by what the examples look like in input space, but by what they demand of the model in parameter space.

The connection to expert specialization is made explicit in the Mixture of Experts framework. Each expert is a separate parameter configuration, and the routing mechanism selects which expert to apply to a given input. If two examples are assigned to the same expert, that expert must accommodate both. If their gradients conflict, the expert will be forced into a compromise; if they align, the expert can specialize without tension. The diversity objective introduced in Chapter 1, that the resulting task vectors should be maximally diverse, is a direct consequence of this reasoning: maximizing the separation between clusters in gradient space minimizes the conflict within each expert and maximizes the distinctness of what each expert learns.

The relationship between gradient similarity and specialization is supported by both theory and experiment. Kawata et al. [7] show that under mild assumptions, a Mixture of Experts trained with stochastic gradient descent can detect latent cluster structure in the data and exploit it to reduce sample complexity, effectively solving a harder problem by decomposing it into easier subproblems. This result is significant because it establishes that the benefit of MoE is not merely architectural but statistical: partitioning the data by its learning structure yields provable gains. On the empirical side, ELREA demonstrates that clustering instruction-tuning data by gradient direction reduces optimization conflicts and improves downstream performance. GradientSpace shows that clustering LoRA gradients and routing to specialized adapters yields improvements over a single fine-tuned model, with the router selecting the appropriate expert based on input features. TAGCOS uses a similar idea for coreset selection, confirming that gradient similarity is a meaningful signal for identifying groups of examples that should be treated together.

In the context of this thesis, gradient similarity plays a dual role. It de-

finer the objective of the clustering step, which is to maximize the similarity of gradients within each bucket and their dissimilarity across buckets. And it justifies the use of a lightweight dispatcher, which approximates the partition by learning to map input states to the clusters that their gradients would induce. The dispatcher does not need to reproduce the clustering exactly, because what matters is not the precise boundary between buckets but the alignment of learning signals within them.

2.3.3 Gradient-Based Clustering

The observation that per-sample gradients encode the learning signal of individual examples leads naturally to the question of how to group examples that share similar learning requirements. Gradient-based clustering answers this by treating the per-sample gradient as a data representation and applying standard clustering algorithms to the resulting set of vectors. Each example is mapped to a point in the gradient space, and clusters correspond to groups of examples whose gradients point in similar directions.

The appeal of this approach lies in its directness. Unlike clustering hidden-layer activations, which groups inputs by what the model already represents, gradient clustering groups examples by what the model needs to change. The partition is therefore aligned with the optimization objective rather than with the current state of the representation. This distinction is what motivates the use of gradient clustering for expert specialization: if the goal is to train separate experts that do not interfere with one another, grouping examples by the direction of their desired updates is a natural way to minimize conflict.

A practical challenge arises from the dimensionality of the gradient space. In modern neural networks, the number of parameters can range from hundreds of thousands to billions, and clustering algorithms that rely on distances or densities degrade as dimensionality increases. The gradient vectors for a single example are as large as the parameter vector itself, and storing them for millions of examples is a non-trivial computational burden. Two strategies are commonly used to address this. The first is to reduce the dimensionality of the gradients before clustering, for example through random projections or principal component analysis. ELREA adopts *random projection* to map gradients to a lower-dimensional space before clustering, trading some information for computational tractability. GradientSpace instead operates directly on the full-dimensional gradient space, using an online *SVD-based algorithm* to identify latent skills without materializing all sample gradients at once. The second strategy is to normalize the gradients before clustering, projecting each vector onto the unit sphere. Normalization removes magnitude information and focuses the clustering on direction, which is the signal most relevant to specialization. This is the approach adopted in this work, and it aligns with the observation that the direction of the update,

not its magnitude, determines whether two examples are compatible.

The empirical evidence for gradient-based clustering is substantial. EL-REA clusters instruction-tuning data by gradient direction and trains a separate LoRA expert on each cluster, reporting reduced optimization conflicts and improved downstream performance compared to a single fine-tuned model. GradientSpace clusters LoRA gradients, trains a specialized expert per cluster, and deploys a lightweight router to select the appropriate expert at inference, achieving consistent gains over clustering and fine-tuning baselines. TAGCOS applies gradient clustering to a different end, using the resulting groups to select a representative coreset for efficient instruction tuning, and demonstrates that a small fraction of the data can retain most of the performance. These works confirm that clustering gradients is not merely a theoretical curiosity but a practical method for identifying structure in training data that is not apparent in the input space.

Theoretical support for the approach comes from Kawata et al. [7], who study the sample and runtime complexity of Mixture of Experts trained with stochastic gradient descent on regression tasks with latent cluster structure. They prove that a vanilla MoE can detect and exploit such structure, effectively decomposing a harder problem into easier subproblems, each associated with an individual cluster. This result provides a formal argument for why partitioning a dataset by its gradient structure should yield statistical benefits: the partition captures the latent cluster structure of the learning problem, and the experts specialize to the subproblems defined by that structure. While the theorem is stated for a specific setting, it reinforces the intuition that gradient similarity is a meaningful signal for grouping examples.

In the context of this thesis, gradient-based clustering is applied not to instruction-tuning data but to chess positions, and not to the full parameter set of a large model but to the head parameters of a small NNUE. The clustering is performed on normalized gradients with respect to the head, and the resulting partition defines the buckets on which the expert heads are trained. The dispatcher then learns to approximate this partition from the L1 activations, enabling fast routing at inference. The remainder of this chapter builds on the principles established here, translating the general idea of gradient clustering into a concrete method that respects the efficiency constraints of embedded evaluation.

2.4 Bucketing in NNUE

2.4.1 Handcrafted Bucketing

In NNUE-based chess engines, the state space is often partitioned into discrete buckets using manually designed rules. Each bucket corresponds to a region of the state space where positions are assumed to share similar char-

acteristics, and a separate set of output weights is trained for each bucket. This approach, sometimes referred to as *phase-based bucketing*, is a pragmatic compromise: it allows the evaluation function to adapt to different types of positions while keeping the per-head network small and fast.

Common features used for bucketing include the total material count (the number of pieces remaining on the board), the presence or absence of specific pieces such as queens or bishops, the location of the kings, and the overall game phase derived from material. For example, Stockfish historically used a phase classification that interpolates between opening and endgame evaluations based on material remaining: positions with many pieces are treated as openings or middlegames, while positions with few pieces are treated as endgames. The Kaggle FIDE & Google Efficient Chess AI Challenge popularised a variant where the game is divided into three phases, opening, middlegame, and endgame, based on material thresholds, with separate evaluation heads for each phase.

These handcrafted bucketing schemes are computationally cheap: the features required for routing are simple integer counts and bitwise operations that add negligible overhead to the evaluation function. They are also interpretable: a chess programmer can inspect the buckets and understand why a position is assigned to a particular region.

However, handcrafted bucketing suffers from fundamental limitations. The features are designed based on human intuition about chess, which may not align with the actual structure of the learning signal. A position in the middlegame with a queen on the board may require a very different adjustment to the evaluation head depending on whether it is a quiet positional struggle or a tactical melee, yet both are assigned to the same bucket based on material count. Conversely, two positions that appear superficially different (for example, an endgame with a rook versus a middlegame with heavy pieces) may require similar adjustments to the head, but are placed in different buckets. In other words, handcrafted features encode what a programmer *thinks* are distinct regimes, not what the model *needs* in order to specialise. This limitation motivates the exploration of data-driven alternatives, where the partition is learned directly from the training signal rather than prescribed by chess expertise.

2.4.2 Learned Bucketing

The limitations of handcrafted bucketing have motivated research into data-driven alternatives, where the partition of the state space is learned directly from the training data rather than prescribed by human intuition. These approaches can be broadly divided into two categories: those that learn the partition from the model’s internal representations, and those that learn a routing policy that selects among experts based on the input state.

Clustering hidden-layer activations

One natural approach is to cluster the activations of the network’s hidden layers, grouping positions that the model already represents similarly. For example, unsupervised concept discovery methods have been applied to decompose the activation space of chess networks such as AlphaZero. The intuition is that positions that produce similar hidden representations are likely to require similar processing from the subsequent layers, making them natural candidates for the same expert head. This approach has the advantage of being fully data-driven and requiring no chess-specific feature engineering. However, it suffers from a fundamental limitation: activations capture what the model *knows*, not what it *needs* to adjust. Two positions may produce similar hidden representations because the model already evaluates them correctly, yet require very different updates to the head; conversely, positions with dissimilar activations may require similar adjustments. The partition is defined by the model’s current state, not by the learning signal that drives specialisation. This limitation is precisely what motivates the gradient-based approach proposed in this thesis.

Learned routing for mixture-of-experts

An alternative to clustering is to learn a routing policy that directly selects among experts based on the input state. In the broader machine learning literature, mixture-of-experts architectures typically employ a learnable gating network that produces a weighted combination of expert outputs. In the chess domain, recent work has explored learned routing for expert selection. **Hexaïssa** [9] formulates expert selection as a MoE problem, learning a gating policy that dynamically selects among heterogeneous engines such as Stockfish. **M2CTS** [10] combines MoE with *Monte Carlo Tree Search* (MCTS), using a modular framework that adapts strategy dynamically based on game phase and achieves a significant increase in engine strength. These approaches demonstrate that learned routing can be effective in chess, but they typically operate at the level of entire engines or large networks, not at the level of lightweight NNUE heads for embedded devices. Moreover, they often rely on **expensive routing mechanisms** that would be prohibitive at every node of an alpha-beta search.

The gap addressed by this thesis

Existing learned bucketing methods for chess either rely on activations (which capture representation, not learning signal) or require computationally expensive routing that is unsuitable for embedded inference. This thesis addresses this gap by proposing a method that learns the partition from sample gradients, the learning signal itself, and then distills this partition into a lightweight linear dispatcher that adds negligible overhead at inference time.

This combines the data-driven advantages of learned bucketing with the efficiency requirements of on-device evaluation, providing a practical alternative to both handcrafted rules and computationally expensive routing policies.

2.5 Related Work

The preceding sections have established the conceptual foundations of this thesis: the architecture of NNUE evaluation, the principles of Mixture of Experts, the role of per-sample gradients as a learning signal, and the landscape of bucketing strategies in chess engines. This final section situates the proposed method within the broader body of work on efficient chess engines, teacher-student evaluation, and gradient-based expert specialization. The goal is not to provide an exhaustive survey, but to clarify how the method presented in Chapter 3 differs from existing approaches and where it inherits from them.

2.5.1 Efficient Chess Engines

The pursuit of efficient chess evaluation has a long history, driven by the need to evaluate millions of positions per second on commodity hardware. Classical engines relied on handcrafted evaluation functions, which were fast but limited in accuracy. The introduction of NNUE marked a turning point by demonstrating that a neural network could be evaluated within the same latency budget, provided that the first layer is sparse and updated incrementally. This principle, exploiting the fact that consecutive positions differ by only a few piece placements, allowed Stockfish and its derivatives to adopt neural evaluation without sacrificing search speed.

The portability of NNUE to constrained environments was demonstrated by Cfish, a C port of Stockfish that strips away the C++ abstractions and compiles to a much smaller binary. Cfish provides multiple evaluation backends, including a pure NNUE mode that eliminates the classical evaluation entirely, making it viable for embedded platforms. It also incorporates SIMD-optimized attack generation for AVX2 and AVX-512, further reducing the cost of move generation and evaluation. Cfish is the closest reference implementation to the target of this thesis, and it serves both as a baseline and as the host engine into which the MoE NNUE is integrated in Chapter 4.

Beyond the choice of architecture, efficiency in chess engines is achieved through a combination of search pruning, reduction techniques, and integer quantization. Modern engines employ a family of pruning methods, including null move pruning, futility pruning, late move reductions, and quiescence search with delta pruning, to reduce the effective branching factor of the alpha-beta search. These techniques are orthogonal to the evaluation function and remain unchanged when the evaluator is replaced. Quantization, by contrast, directly concerns the evaluation function: NNUE networks are

designed for low-precision integer inference, using int8 and int16 operations to exploit the available hardware performance of modern CPUs. The quantization process introduces some approximation error, but for the shallow architectures typical of NNUE, this error is negligible. In Chapter 4, we adopt the same quantization strategy when deploying the MoE NNUE on the Wio Terminal, ensuring that the additional expert heads and dispatcher fit within the memory and latency budget of the device.

What distinguishes this thesis from the efficient-engine literature is not the quest for raw speed, but the question of how to allocate a fixed parameter budget more effectively. The engines discussed above use a single head to evaluate all positions; the MoE architecture distributes the head parameters across multiple experts and adds a dispatcher to route each position to the appropriate expert. The dispatcher is designed to cost almost nothing at inference time, so the efficiency of the evaluation function is preserved. Whether the specialization gained from this partition outweighs the additional memory footprint is the central empirical question of Chapter 5.

2.5.2 Teacher–Student Evaluation

The idea of training a compact model to imitate a stronger or more expensive one has a long history in machine learning, formalised most influentially by Hinton et al. [11] under the name of knowledge distillation. In the original formulation, a large teacher network produces soft probability distributions over classes, and a smaller student network is trained to match those distributions rather than the hard labels of the training data. The soft targets carry more information than one-hot labels because they encode the teacher’s uncertainty and the relative similarity between classes, and this **richer signal** allows the student to achieve better generalisation than it would from labels alone.

In the chess domain, teacher–student evaluation has become the dominant paradigm for training evaluation functions. AlphaZero [3] used a form of self-distillation: the same network served as both policy and value function, and the targets for training were generated by Monte Carlo Tree Search guided by the network itself. This created a feedback loop in which the search improved the policy, and the improved policy trained a better network, which in turn guided a stronger search. Lc0, the open-source implementation of the AlphaZero approach, follows the same principle and has produced some of the strongest chess networks available. Its value head outputs a probability distribution over win, draw, and loss, making it directly compatible with the soft cross-entropy objective used in this work.

A different teacher–student setup is used in the training of Stockfish’s NNUE. The teacher is not a separate network but the engine itself, run at a shallow search depth on a large corpus of positions. The resulting centipawn evaluations, or in newer versions the WDL probabilities derived from the

search, serve as targets for the NNUE. This approach is sometimes described as self-distillation because the teacher and student share the same source of knowledge, but it differs from Lc0 in that the teacher is a search process rather than a fixed neural network.

The choice of the teacher has direct consequences for the training signal. A neural teacher such as Lc0 provides smooth, calibrated probability distributions that reflect its internal uncertainty, while a search-based teacher provides sharper targets that may be more accurate tactically but less informative about positional nuance. In this thesis, we adopt Lc0 as the teacher for two reasons. First, its WDL output aligns naturally with the three-way output of the NNUE value head, eliminating the need to convert centipawn evaluations into probabilities. Second, its strength makes it a reliable source of labels across all phases of the game. The labelling pipeline, including the choice of search depth and the treatment of label noise, is described in Section 4.1.4.

What distinguishes this thesis from the teacher–student literature is not the use of distillation itself, which is standard practice, but the way in which the student is partitioned. Rather than training a single student to match the teacher on all positions, we train multiple students, each specialised on a distinct region of the state space, sharing a common representation but with separate heads. The teacher remains the same for all experts, and the distillation objective is unchanged. The specialisation arises from the partition of the training data, not from a change in the target. Whether this partition yields a more accurate student than a single model trained on all data is the empirical question addressed in Chapter 5.

2.5.3 Gradient-Based Expert Specialization

The works most directly related to this thesis are those that use gradient information to partition training data and train specialized experts on the resulting groups. These approaches share a common pipeline: compute per-sample gradients, cluster them, train an expert on each cluster, and deploy a routing mechanism to select the appropriate expert at inference. The differences lie in the domain, the type of expert, and the routing strategy.

ELREA [1] partitions instruction-tuning data by gradient direction and trains a separate LoRA expert on each partition. The motivation is to reduce optimization conflicts: when examples in a batch demand opposing parameter updates, the resulting gradient is a compromise that benefits none of them. By grouping examples with aligned gradients, ELREA ensures that each expert sees a coherent learning signal. At inference, the method routes to a weighted ensemble of experts based on gradient similarity, requiring on-the-fly gradient computation for the input. This makes inference expensive and unsuitable for latency-critical applications.

GradientSpace [2] follows a similar pipeline but addresses the inference

cost by training a lightweight encoder-based router. The router predicts the appropriate expert from the input features alone, eliminating the need for gradient computation at inference time. The method clusters LoRA gradients using an online SVD-based algorithm that avoids materializing all sample gradients at once, and trains a separate LoRA expert per cluster. The router is a small encoder that maps input features to a cluster index. This is the closest methodological precedent for the work presented in this thesis: the pipeline of gradient clustering, expert fine-tuning, and lightweight routing is the same. The differences are domain-specific and architectural.

TAGCOS [8] applies gradient clustering to coreset selection rather than expert training. It clusters per-sample gradients and selects a representative subset from each cluster, reducing the amount of data needed for fine-tuning while preserving the diversity of the learning signal. While the end goal is different, the underlying principle is the same: gradient similarity identifies groups of examples that can be treated together. TAGCOS is relevant because it confirms that gradient-based grouping is a robust signal for data selection, not just for expert specialization.

Gradient Atoms [12] takes a different approach, decomposing per-document gradients into sparse components via dictionary learning in a pre-conditioned eigenspace. The goal is behaviour discovery and attribution rather than expert training, but the work reinforces the idea that gradients contain interpretable structure that can be exploited for downstream tasks.

Several differences distinguish this thesis from these prior works. The first concerns the type of expert. In the LLM setting, experts are implemented as LoRA adapters, which are low-rank modifications to a large frozen model. In this work, experts are separate instances of a small NNUE head, trained from the same initialization and sharing a frozen L1 accumulator. LoRA is unnecessary here because the base model is already tiny, and the head that is specialized is only a few thousand parameters. The second difference concerns the routing mechanism. ELREA requires gradient computation at inference, which is prohibitive for a chess engine. GradientSpace uses a small encoder, which is more efficient but still larger than what is feasible on a microcontroller. This thesis uses a linear dispatcher on the L1 activations, which adds only a matrix-vector multiplication of negligible size. The third difference concerns the domain. The prior works target instruction-following in language models, where the target is a distribution over tokens. This thesis targets value estimation in chess, where the target is a WDL distribution from a teacher engine, and the constraints are those of an embedded device rather than a server.

A further difference is the treatment of gradient magnitude. ELREA and GradientSpace operate on full-dimensional or randomly projected gradients, retaining magnitude information. This thesis normalizes each gradient vector to the unit sphere before clustering, focusing the partition on direction rather than magnitude. The motivation is that the direction of the update

determines whether two examples are compatible, while the magnitude is sensitive to the current loss value and position difficulty. Normalization also makes the clustering more robust to outliers and aligns with the cosine-distance objective used in the diversity metric.

In summary, this thesis adapts the gradient-based expert specialization paradigm to a new domain and a new set of constraints. The core idea, cluster by gradient direction, train experts on the clusters, and route at inference, is shared with ELREA and GradientSpace. The contributions are the adaptation to NNUE, the use of head-parameter gradients rather than LoRA gradients, the linear dispatcher on frozen L1 activations, and the validation on a resource-constrained chess engine. These differences are not incremental; they reflect the distinct requirements of embedded inference, where every additional parameter and every additional operation has a measurable cost.

Chapter 3

Method: Unsupervised Bucketing via Sample Gradients

3.1 Method Overview

This chapter presents the proposed method for unsupervised state-space bucketing via sample gradients. The central idea is to use the learning signal itself, the per-sample gradients of the loss with respect to the head parameters, as the basis for partitioning the state space, rather than relying on handcrafted features or hidden-layer activations.

The method unfolds through a carefully orchestrated sequence designed to balance computational tractability with the stringent efficiency demands of on-device inference. It begins by training a base NNUE model on the complete dataset, which establishes a shared L1 representation and provides a stable reference point for subsequent gradient computations. From this foundation, normalized per-sample gradients are extracted with respect to the trainable head parameters, effectively capturing the direction in which each position would push the head during optimization.

These gradient vectors then become the substrate for clustering, yielding a partition of the state space into B distinct buckets that group together positions exhibiting similar learning dynamics. To enable fast routing at inference time without the prohibitive cost of recomputing gradients, a lightweight linear dispatcher is trained to predict bucket assignments directly from the frozen L1 activations.

The pipeline culminates in fine-tuning a dedicated expert head on each bucket, initialized from the base model while keeping the L1 weights frozen. The end result is a mixture-of-experts NNUE architecture in which each expert develops specialised competence over a coherent region of the state space, guided by a dispatcher that introduces negligible overhead to the

evaluation function.

3.2 Notation and Definitions

We introduce here the formal notation used throughout this chapter and the remainder of the thesis. The notation is organized into sets, scalars, vectors and matrices, functions, and key operators.

3.2.1 Sets

Symbol	Description
$\mathcal{S}$	The space of all possible chess positions (states).
$\mathcal{D}$	The training dataset of positions with labels: $\mathcal{D} = \{(s_i, v_i)\}_{i=1}^N$.
$\mathcal{P} = \{\mathcal{D}_1, \dots, \mathcal{D}_B\}$	A partition of the state space into B buckets, where each $\mathcal{D}_i \subset \mathcal{S}$ is a non-empty subset.
Θ	The space of all model parameters (weights).

3.2.2 Scalars

Symbol	Description
B	The number of experts (buckets).
N	The total number of positions in the training dataset.
N_i	The number of positions in bucket $\mathcal{D}_i$.
W	The hidden dimension of the NNUE accumulator per perspective (128 in this work).
H	The hidden dimension of the L2 layer (256 in this work).
d_{in}	The input dimension of the NNUE accumulator (844 in this work).
$v_i \in \mathbb{R}$	The scalar label (expected reward) for position s_i .
η	The learning rate used for gradient updates.

3.2.3 Vectors and Matrices

Symbol	Type	Description
$W_{L1} \in \mathbb{R}^{d_{in} \times W}$	Matrix	Weights of the first layer (accumulator), frozen after base training.
$W_{L2} \in \mathbb{R}^{2W \times H}$	Matrix	Weights of the second layer.
$W_{out} \in \mathbb{R}^{H \times 3}$	Matrix	Weights of the output layer (3 logits for WDL).
$w_{base} \in \mathbb{R}^P$	Vector	All parameters of the base model: $w_{base} = \text{vec}(W_{L1}, W_{L2}^{base}, W_{out}^{base})$.
$\theta_i \in \mathbb{R}^P$	Vector	All parameters of the expert model i : $\theta_i = \text{vec}(W_{L1}, W_{L2}^{(i)}, W_{out}^{(i)})$.
$\delta_i \in \mathbb{R}^{P_{head}}$	Vector	The task vector for expert i : $\delta_i = \theta_i^{head} - \theta_{base}^{head}$, where $\theta^{head} = \text{vec}(W_{L2}, W_{out})$.
$\Delta_i \in \mathbb{R}^{P_{head}}$	Vector	The per-sample gradient for position s_i : $\Delta_i = \nabla_{w_{head}} \mathcal{L}(\hat{f}_{w_{base}}(s_i), v_i)$.
$x_i \in \{0, 1\}^{d_{in}}$	Vector	The sparse binary feature vector for position s_i .
$h_i = W_{L1} \cdot x_i \in \mathbb{R}^W$	Vector	The accumulator output (L1 activation) for position s_i .

3.2.4 Functions and Models

Symbol	Description
$f_w : \mathcal{S} \rightarrow \mathbb{R}^3$	The NNUE evaluation function parameterized by weights w , producing WDL logits.
$\hat{f}_w : \mathcal{S} \rightarrow \mathbb{R}$	The NNUE scalar value function: $\hat{f}_w(s) = \text{softmax}(f_w(s))_W - \text{softmax}(f_w(s))_L$.
$g_\phi : \mathcal{S} \rightarrow \{1, \dots, B\}$	The dispatcher function, parameterized by ϕ , mapping a position to a bucket index.
$\mathcal{L}(y, v)$	The loss function, typically soft cross-entropy on WDL probabilities.
$\mathcal{L}_{acc}(y, v)$	The accumulated loss over a batch or epoch.

3.2.5 Key Operators

Symbol	Description
$\text{vec}(\cdot)$	The vectorization operator, flattening a matrix into a column vector.
$\nabla_w \mathcal{L}$	The gradient of the loss with respect to the parameters w .
$\ \cdot\ $	The Euclidean (L2) norm.
$\text{softmax}(z)_k = \frac{e^{z_k}}{\sum_j e^{z_j}}$	The softmax function over logits z .
$d_{\cos}(u, v) = 1 - \frac{u \cdot v}{\ u\ \ v\ }$	The cosine distance between two vectors u and v .

3.2.6 Relationship Between Δ_i and δ_i

A crucial distinction in this work is between **per-sample gradients** Δ_i and **task vectors** δ_i . These two concepts are similar to each other, but it is worth emphasizing that the task vector $\delta_i = \theta_i - \theta_{\text{base}}$ is the difference in model parameters between before and after the fine-tuning, while the sample gradient $\Delta_i = \nabla_{w_{\text{head}}} \mathcal{L}(\hat{f}_{w_{\text{base}}}(s_i), v_i)$ is the gradient of the loss with respect to the head parameters, evaluated at a single position s_i using the base model.

The central hypothesis of this work is that clustering positions by their Δ_i yields a partitioning for which the resulting learning dynamics are maximally diverse, each expert specializes in a distinct region of the state space, and that the dispatcher is able to approximate this partitioning with enough accuracy to preserve its general structure.

3.3 Step 1: Train the Base Model

The first step of our method is to train a **base evaluation model** that will serve as the foundation for all subsequent steps. This model provides two essential functions: it supplies the reference point from which the sample gradients are computed, and it provides the frozen L1 representation used by the dispatcher at inference time.

3.3.1 Model Architecture

The base model follows the NNUE architecture described in Section 2.1.3: a sparse accumulator layer W_{L1} that maps a binary feature representation to a hidden state in $\mathbb{R}^W$, followed by a small fully-connected head (W_{L2}, W_{out}) that produces WDL logits. The architecture is kept deliberately small to reflect the resource constraints of the target hardware, specifically a hidden dimension of $W = 128$ for the accumulator and $H = 256$ for the L2 layer, resulting in approximately 175,000 trainable parameters.

3.3.2 Training Objective

The model is trained to minimize the **soft cross-entropy loss** between its predicted WDL distribution and the teacher labels provided by Lc0 (see Section 4.1.4). For a batch of N positions with teacher probabilities $\hat{p}_i = (\hat{P}_i(W), \hat{P}_i(D), \hat{P}_i(L))$ and model outputs $p_i = (P_i(W), P_i(D), P_i(L))$, the loss is:

$$\mathcal{L} = -\frac{1}{N} \sum_{i=1}^N \left[\hat{P}_i(W) \log P_i(W) + \hat{P}_i(D) \log P_i(D) + \hat{P}_i(L) \log P_i(L) \right].$$

This choice of loss is deliberate. Unlike mean squared error on a scalar value (centipawns or expected reward $v = P(W) - P(L)$), the cross-entropy loss encourages the model to match the **full outcome distribution** rather than just its mean. This provides a richer training signal and naturally handles the non-linear relationship between WDL probabilities and the scalar evaluation used during search.

3.3.3 Training Protocol

The base model is trained on the full dataset of approximately 105 million positions using the Adam optimiser with a learning rate of 10^{-2} , linearly decayed to 10^{-3} over the course of training. We use a batch size of 10,000 and train until convergence on the held-out test set (1% of the total dataset). All training is performed in PyTorch on a DGX Nvidia Spark GPU.

3.3.4 The Role of the Base Model

Once trained, the base model $w_{\text{base}} = (W_{L1}, W_{L2}^{\text{base}}, W_{\text{out}}^{\text{base}})$ serves several critical functions in our pipeline, one of which is as reference point for gradients. For each position s_i , we compute the sample gradient Δ_i evaluated at the base model. This gradient represents the direction in which the head would need to move to better fit that specific position, the *learning signal* that drives our bucketing.

Another important role of the base model is as frozen representation for routing. The L1 weights W_{L1} are frozen after base training and are never updated during expert fine-tuning. This ensures that all experts operate on the same shared representation, and that the dispatcher can rely on stable L1 activations $h = W_{L1} \cdot x$ as input features.

Finally, the base model is of course also the starting point for expert fine-tuning. Each expert head $(W_{L2}^{(i)}, W_{\text{out}}^{(i)})$ is initialised from the base head $(W_{L2}^{\text{base}}, W_{\text{out}}^{\text{base}})$ before being fine-tuned on its assigned bucket.

3.3.5 Why Freeze L1?

Freezing the L1 layer is a deliberate design choice. The accumulator is the most expensive component of the NNUE architecture in terms of parameter count, and updating it during fine-tuning would be computationally prohibitive. More importantly, freezing L1 ensures that the representation space remains the same for all experts: the dispatcher, trained on L1 activations, can reliably route positions without needing to account for different representations. This stability is essential for the lightweight inference pipeline, where the dispatcher must operate with negligible overhead.

3.4 Step 2: Compute Sample Gradients

With the base model trained, the second step is to compute, for each position in the dataset, the sample gradient of the loss with respect to the head parameters. These gradients encode the direction in which the head parameters would need to move to improve the prediction for each individual position. They represent the *learning signal* that we will later use for bucketing.

3.4.1 Definition of Sample Gradient

For each position s_i in the dataset $\mathcal{D} = \{(s_i, v_i)\}_{i=1}^N$, the sample gradient is defined as:

$$\Delta_i = \nabla_{w_{\text{head}}} \mathcal{L}(\hat{f}_{w_{\text{base}}}(s_i), v_i),$$

where $w_{\text{head}} = \text{vec}(W_{L2}, W_{\text{out}})$ is the vector of all head parameters (the L2 weights and biases, and the output weights and biases), $\hat{f}_{w_{\text{base}}}$ is the base model evaluated at the current weights, $\mathcal{L}$ is the soft cross-entropy loss on WDL probabilities, and $v_i = (p_W, p_D, p_L)$ is the teacher label for position s_i .

The gradient is computed at the base model w_{base} , before any fine-tuning occurs. It represents the instantaneous direction in weight space that would reduce the loss on that specific position, independent of all other positions.

3.4.2 Implementation

In practice, the gradient computation leverages PyTorch’s `torch.autograd.grad` function, which efficiently computes sample gradients for an entire batch of inputs in a single operation. Crucially, these gradients are computed only with respect to the head parameters, namely (W_{L2}, W_{out}) , while the L1 accumulator weights remain excluded from this differentiation.

The workflow begins with a forward pass through the model for a batch of positions, producing WDL predictions that are then compared against teacher labels via cross-entropy loss. Calling `torch.autograd.grad` on the loss with respect to the head parameters yields the desired gradients, which are subsequently detached from the computation graph and flattened into a single vector representation for each position. This entire computation runs fully parallelised on the GPU and completes in a single pass over the dataset, with the resulting gradient tensors persisted to disk for use in the downstream clustering stage.

3.4.3 Normalisation

The raw gradients can have highly variable magnitudes depending on the position, the current state of the model, and on whether the multiplicity of the state is considered or not. The *L2 normalization* of each gradient

vector has been shown to work well in gradient-clustering literature [1, 2] and preserves the relative angular structure of the gradients:

$$\Delta_i^{\text{norm}} = \frac{\Delta_i}{\|\Delta_i\| + \epsilon}.$$

This projects each gradient onto the unit hypersphere, preserving direction while removing magnitude information. This is appropriate because the *direction* of the gradient encodes the type of specialisation needed, while the magnitude is more sensitive to the current loss value and position difficulty.

3.4.4 Storage and Compute Considerations

Computing and storing sample gradients for 105 million positions presents practical challenges. Each gradient vector has dimension $P_{\text{head}} \approx 66,563$ (flattened L2 and output weights and biases). Storing this as 32-bit floats would require approximately

$$105 \times 10^6 \times 66,563 \times 4 \text{ bytes} \approx 28 \text{ TB}.$$

The implementation chapter describes how this cost is managed in practice through half-precision storage, memory mapping, and, when appropriate, computation on a subset of the data.

3.4.5 Handling Duplicate Positions

In a dataset of chess positions extracted from human games, the same position can occur many times. Opening positions, in particular, are heavily overrepresented: the starting position appears in every game, and the first few moves are shared across a large fraction of the dataset. This multiplicity has two distinct effects on the pipeline, one on training and one on clustering, and they call for different treatments.

During training, the frequency of a position determines its contribution to the loss. If every occurrence is treated equally, the natural distribution of the data is preserved: common positions contribute more, rare positions contribute less. This is statistically appropriate if the goal is to match the distribution of positions encountered during play. However, extreme overrepresentation is wasteful, both computationally and statistically: the starting position carries no more information on its millionth occurrence than on its first, and it can dominate the gradient signal to the detriment of more informative positions. A common remedy is to cap the multiplicity of any position, so that no single state contributes more than a fixed number of times per epoch.

For clustering, the issue is different. The clustering operates on sample gradients, and duplicated positions produce identical gradient vectors. If

duplicates are retained, they occupy a disproportionate volume in the gradient space, and cluster centroids are pulled toward common positions. This biases the partition toward regions of the state space that are frequent in the data, rather than regions that require distinct learning signals. Deduplicating before clustering avoids this bias, but it discards the information that some positions are more important than others.

3.5 Step 3: Cluster Sample Gradients

With the normalised sample gradients $\{\Delta_i^{\text{norm}}\}_{i=1}^N$ computed for every position in the dataset, the third step is to partition the data into B clusters (buckets) such that positions with similar learning signals are grouped together. This partition will define the assignment of positions to expert heads during fine-tuning.

3.5.1 The Objective of Clustering

Recall the central hypothesis of this work: clustering positions by their sample gradients Δ_i yields a partition $\mathcal{P} = \{\mathcal{D}_1, \dots, \mathcal{D}_B\}$ for which the resulting task vectors $\delta_i = \theta_i - \theta_{\text{base}}$ are maximally diverse. The role of the clustering algorithm is to discover a partition that approximates this objective. From a practical standpoint, we seek an efficient clustering algorithm that produces clusters that are cohesive, balanced, stable, and, hopefully, as separated as possible. Different clustering algorithms make different trade-offs with respect to these criteria. We consider two broad families: fixed- B algorithms and density-based algorithms.

3.5.2 Fixed- B Algorithms: K-Means and Variants

The most widely used clustering algorithm is **K-Means**, which partitions the data into B clusters by minimising the within-cluster sum of squares. For a set of clusters $\{\mathcal{C}_1, \dots, \mathcal{C}_B\}$, K-Means minimises:

$$\sum_{k=1}^B \sum_{i \in \mathcal{C}_k} \|\Delta_i - \mu_k\|^2,$$

where $\mu_k = \frac{1}{|\mathcal{C}_k|} \sum_{i \in \mathcal{C}_k} \Delta_i$ is the centroid of cluster k . This algorithm is fast and well-understood. Specifically, *Mini-Batch K-Means* is particularly attractive for our scale, as it efficiently handles large datasets by processing data in mini-batches, making it suitable for our dataset, that comprises millions of chess positions. It reduces memory requirements and converges faster than standard K-Means, while producing nearly identical results.

3.5.3 Density-Based Algorithms

An alternative family of algorithms, **density-based clustering**, does not require a fixed number of clusters. Instead, these methods identify clusters as regions of high density separated by regions of low density.

Density Peak Clustering [13] is a particularly relevant algorithm for our setting. It works by identifying cluster centres as points that have high local density (many neighbours within a cutoff distance) and are far from points with higher density (suggesting they are local maxima).

The number of clusters emerges naturally from the data: each point with density higher than all its neighbours and with large distance to the nearest higher-density point is a cluster centre. The remaining points are assigned to the same cluster as their nearest higher-density neighbour.

We chose to use this algorithm because it automatically determines B , so it will be interesting to find out what the clusters of board positions actually represent. This algorithm is also notoriously robust to outliers; points with low density and large distance to higher-density points are naturally identified as outliers.

DBSCAN (Density-Based Spatial Clustering of Applications with Noise) defines clusters as regions of high density separated by regions of low density, using two parameters: ε (neighbourhood radius) and a minimum number of points to form a dense region. It does not require the number of clusters B as an input and can label outliers as noise. It avoids the need to pre-specify B , which could be useful when the natural structure of the gradient space is unknown. It can detect clusters of arbitrary shapes and is robust to outliers, potentially identifying positions that yield uninformative gradients.

3.5.4 Choosing B and the Algorithm

The choice between fixed- B and density-based clustering depends on the relative importance of architectural efficiency and data-driven discovery. A density-based approach that automatically determines B could reveal the natural structure in the gradient space, potentially identifying a number of clusters that better reflects the underlying distribution of learning signals. On the other hand, hardware constraints might impose a hard limit on how many expert heads our device is allowed to store based on our architecture of choice.

3.5.5 Validation and Diagnostics

Once the clustering is complete, the quality of the resulting partition is assessed using several complementary metrics. The purpose of this step is not to select a single best partition, but to understand the structure of the gradient space and to guide the choice of the number of experts B used in the subsequent fine-tuning stage.

The **cluster size distribution** provides a first diagnostic. We examine the number of positions assigned to each bucket to ensure that no cluster is too small to support stable fine-tuning. A bucket with very few positions may lead to an expert head that is poorly conditioned or that overfits its small training set. Conversely, a highly imbalanced distribution may indicate that the clustering algorithm has collapsed most of the data into a single region, which would defeat the purpose of partitioning. A perfectly balanced distribution is not required, but the sizes should be large enough to train each expert reliably.

Inertia, the within-cluster sum of squared distances to the centroid, measures the compactness of the clusters. For a fixed value of B , lower inertia indicates tighter clusters. We compute inertia across a range of B values and look for an elbow point, the value beyond which additional clusters yield diminishing reductions in inertia. This heuristic is not definitive, but it provides a useful indication of the natural structure of the gradient space and helps constrain the range of B values worth exploring.

The **silhouette score** [14] measures how similar each point is to its own cluster compared to the nearest neighbouring cluster. For a point i , the coefficient is defined as

$$s_i = \frac{b_i - a_i}{\max(a_i, b_i)},$$

where a_i is the mean distance to the other points in the same cluster and b_i is the mean distance to the points in the nearest other cluster. Values range from -1 to $+1$, with higher values indicating better-defined clusters. A score near zero suggests overlapping clusters, while negative values indicate that some points may be assigned to the wrong cluster. We report the average silhouette score over all positions, but interpret it with caution: the gradient space is high-dimensional, and silhouette scores tend to degrade as dimensionality increases even when the data exhibits meaningful structure.

Finally, the **cosine distance between cluster centroids** provides a direct measure of the diversity that is central to the method. The objective of the partition is to obtain task vectors $\delta_i = \theta_i - \theta_{\text{base}}$ that are maximally diverse, and the pairwise cosine distance between centroids serves as a proxy for this diversity. We compute the cosine distance between the centroid of clusters in the normalised gradient space. High average distances indicate that the clusters are well separated and likely to induce distinct task vectors. Low distances suggest that the clusters are redundant and may not yield meaningful specialisation. We report both the average and the minimum pairwise distance, since the presence of even one nearly collinear pair of centroids may indicate that two buckets could be merged without loss of diversity.

Together, these diagnostics provide a comprehensive view of the partition and inform the choice of B . In practice, the clustering pipeline is run for a range of values, such as $B \in \{2, 4, 8, 16, 32\}$, and the partition that offers the

best balance between compactness, separation, and cluster size is selected. The chosen partition is then used for dispatcher training and expert fine-tuning in the subsequent steps.

3.6 Step 4: Train the Dispatcher

The clustering step yields a partition of the training dataset into B buckets, but this partition is defined in terms of sample gradients. At inference time, gradients are unavailable; computing them would require the teacher evaluation of the position, which we of course do not have. To route a new position to the appropriate expert, we therefore need a mechanism that predicts the bucket assignment from features that are already computed during the normal forward pass. The dispatcher g_ϕ serves this purpose.

3.6.1 Architecture

The dispatcher is a linear classifier that takes as input the concatenated L1 activations $h = [h_{\text{own}} || h_{\text{opp}}] \in \mathbb{R}^{2W}$ and produces a vector of logits over the B buckets:

$$z = W_{\text{disp}} h + b_{\text{disp}},$$

where $W_{\text{disp}} \in \mathbb{R}^{B \times 2W}$ and $b_{\text{disp}} \in \mathbb{R}^B$. During training, a softmax function converts these logits into a probability distribution over buckets, and the model is trained to predict the cluster assignments produced by the clustering step. At inference, the softmax is discarded and the predicted bucket is simply the argmax of the logits:

$$g_\phi(s) = \arg \max_i z_i.$$

This design ensures that routing adds only a matrix-vector multiplication and a comparison, both of which are inexpensive integer operations.

3.6.2 Why a Linear Model

The choice of a linear dispatcher is deliberate and follows directly from the inference-time constraints discussed in Section 2.1.5. A linear layer with $2W$ inputs and B outputs requires $2W \times B + B$ parameters, which for $W = 128$ and $B = 8$ amounts to approximately 2,056 parameters. This is small enough to fit comfortably in the flash memory of a microcontroller alongside the expert heads. The computation itself is a single matrix-vector product, which can be implemented with integer arithmetic and adds negligible latency to the evaluation function.

A more expressive dispatcher, such as a multi-layer perceptron with hidden layers, could potentially achieve higher classification accuracy. However, the additional parameters and non-linearities would increase both memory

footprint and inference cost. Given the extreme efficiency requirements of the target hardware, we prioritize simplicity and speed over marginal gains in routing accuracy. As we discuss in Section 3.6.4, the linear dispatcher is sufficient to capture the coarse structure of the partition.

3.6.3 Training

The dispatcher is trained on the same dataset used for clustering, with the cluster labels serving as targets. The input features are the L1 activations $h_i = W_{L1} \cdot x_i$, which are already computed during the base model’s forward pass and can be cached for the entire dataset. The loss is the standard cross-entropy between the predicted distribution and the one-hot cluster assignment:

$$\mathcal{L}_{\text{disp}} = -\frac{1}{N} \sum_{i=1}^N \log(g_\phi(s_i)_{c_i}),$$

where c_i is the cluster index assigned to position s_i . We optimize this loss using Adam with a learning rate of 10^{-2} , decayed to 10^{-3} over the course of training. Training typically converges within a few epochs, and we use early stopping on a held-out validation set to prevent overfitting. Because the dispatcher is a small linear model, the entire training procedure takes only a few minutes on a GPU.

3.6.4 Can a Linear Dispatcher Capture the Clustering?

The sample-gradient space is highly dimensional ($P_{\text{head}} \approx 66,563$), while the dispatcher operates on the L1 activations, which have dimension $2W = 256$. A natural question is whether a linear function of these activations can accurately predict the cluster assignments derived from gradients. The answer depends on how much information about the learning signal is already encoded in the L1 representation.

3.6.5 Fixed vs. Re-Assigned Cluster Assignments

A subtle but important design decision concerns the relationship between the dispatcher and the expert fine-tuning. During the fine-tuning step (Section 3.7), each position is assigned to a bucket. We can either use the original cluster assignments produced by the clustering algorithm, or we can re-assign positions using the dispatcher’s predictions. These two options have different implications.

If we use the dispatcher to re-assign positions, the experts are trained on the buckets that the dispatcher believes are correct, rather than the buckets discovered by clustering. This could potentially align the experts more closely with the dispatcher’s behavior at inference time. The downside is

that some of the structure and complexity of the original clustering is lost, in favor of a simpler, linear partitioning.

3.7 Step 5: Fine-Tune Expert Heads

The final step of the pipeline produces the specialised experts. Once the clustering has defined the partition and the dispatcher has been trained to approximate it, we fine-tune a separate head on each bucket, starting from the base model and keeping the L1 representation frozen. The result is a set of B expert heads, each adapted to a distinct region of the state space, sharing a common L1 accumulator.

3.7.1 What Is Trained and What Is Frozen

The L1 accumulator remains frozen throughout this step, as it has been since the base model was trained. This is not merely a convenience but a structural requirement: the dispatcher relies on L1 activations to route positions, and those activations must be computed by the same weight matrix regardless of which expert is selected. If each expert were to fine-tune its own L1 weights, the dispatcher would need to account for B different representation spaces, and the inference pipeline would become unworkable. Freezing L1 also keeps the memory footprint constant: all experts share the same accumulator, and only the head parameters differ between them.

Each expert head consists of the L2 layer and the output layer, parameterised by $(W_{L2}^{(i)}, b_{L2}^{(i)}, W_{out}^{(i)}, b_{out}^{(i)})$. These are initialised from the corresponding parameters of the base model, so that every expert starts from the same well-trained foundation. The only thing that distinguishes one expert from another is the data on which it is fine-tuned, the subset of positions assigned to its bucket by the clustering step.

3.7.2 Training Procedure

For each bucket $\mathcal{D}_i$, we fine-tune a copy of the base head on the positions in that bucket, using the same soft cross-entropy loss and optimiser as in the base training. The L1 activations are pre-computed once for the entire dataset and cached, so the fine-tuning step only requires forward and backward passes through the small head, not the full model. This makes the procedure extremely fast: fine-tuning B heads on subsets of a dataset of millions of positions takes only a fraction of the time required to train the base model.

We use a short training schedule, typically one or two sweeps over the bucket, with early stopping based on the cross-entropy on a held-out portion of the bucket. Depending on the dataset size, one of two approaches is best. If the buckets are too small, training for too long risks overfitting. In our

specific case data is abundant, and the goal is to reach the best possible performance on each bucket in isolation, to produce a set of experts whose combined behaviour significantly improves upon the single base head. The learning rate is set lower than in base training, to avoid large deviations from the base head that could destabilise the shared L1 representation.

3.7.3 Data Availability per Expert

Partitioning the dataset into B buckets means that each expert sees only a fraction of the total data. If the partition is balanced, each expert is fine-tuned on approximately N/B positions. For $B = 8$ and $N = 105$ million, this amounts to roughly 13 million positions per expert, which is still a substantial training set. However, if the clustering produces imbalanced buckets, some experts may be trained on far fewer positions, which can lead to underfitting or unstable training.

This is one of the reasons why we monitor the cluster size distribution as part of the validation diagnostics in Section 3.5.5. If a bucket is too small to support stable fine-tuning, several remedies are possible, and we return to this question in the experimental evaluation.

3.7.4 The Resulting MoE Model

At the end of this step, we have a complete mixture-of-experts evaluation function. The model consists of three components: the frozen L1 accumulator, which is shared across all experts; the dispatcher, which maps L1 activations to a bucket index; and the B expert heads, each containing its own L2 and output parameters. During inference, a position is encoded into its sparse feature representation, passed through L1 to obtain the accumulator vector, routed by the dispatcher to a single bucket, and finally evaluated by the corresponding expert head. The output is a WDL distribution from the side-to-move perspective, from which the scalar evaluation is derived as usual.

The inference cost is therefore one L1 forward pass (which is incremental during search), one linear dispatcher operation, and one head forward pass. Compared to the single-head base model, the only additional cost is the dispatcher, which as we have seen adds a matrix-vector multiplication of negligible size. The experts themselves are not more expensive than the base head: they have the same architecture, and only one is evaluated per position. The memory cost is B times the size of the head parameters, which remains small relative to the L1 accumulator. This is the essential trade-off of the method: we gain specialisation at the cost of additional head parameters, while keeping the inference path as lean as the base model.

3.8 Generalisation Beyond Chess

The method presented in this chapter is not specific to chess. Its core ingredients are a state space, a parametric model that maps states to predictions, a differentiable loss, and a dataset of *state-target* pairs. Given these, the pipeline can be applied to a different domain without modification: train a base model, compute per-sample gradients with respect to the parameters of a head, cluster them, train a dispatcher on a frozen intermediate representation, and fine-tune specialised heads on the resulting buckets. Chess is an attractive testbed because it offers a large dataset of labelled positions, a well-established efficient architecture in NNUE, and clear resource constraints that make the efficiency of the dispatcher meaningful. But the underlying principle, that a partition of the state space can be discovered by clustering the learning signal rather than the input or the representation, is domain-agnostic.

Several other settings share the structural properties that make the method applicable. In similar board games such as shogi, the same formulation carries over directly: shogi engines already use NNUE-style accumulators, and the position encoding and WDL labels can be adapted with minimal changes. In the ancient game of Go, the NNUE architecture should be swapped for a model of convolutional nature. In video game AI and simulated environments, where agents must evaluate states or select actions under tight latency budgets, a similar decomposition into a shared representation and a small routed head could reduce inference cost while preserving specialisation. In robotics, where control policies often operate on high-frequency sensor streams and must run on embedded hardware, the same pattern of a frozen feature extractor followed by a lightweight, routed head is a natural fit: the feature extractor runs continuously, while the head is selected by a dispatcher that observes the same features at negligible cost.

The method also connects to recent work on world models, where a learned representation of the environment is used to predict future states or plan actions. World models typically maintain a latent state that is updated incrementally as the environment evolves, much like the NNUE accumulator is updated as pieces move. This parallel suggests that the gradient-based bucketing idea could be extended to world models, where different regions of the latent state space may require different dynamics or reward predictors. In such a setting, the head would be the component that predicts the next latent state or the reward, and the dispatcher would route to a specialised predictor based on the current latent representation. The efficiency argument carries over: a dispatcher that operates on the shared latent state adds minimal cost compared to evaluating multiple full predictors.

What must change across domains is not the algorithmic structure but the concrete choices within it. The feature encoding, the architecture of the base model, the loss function, and the definition of the target all depend on

the domain. In chess, we use a sparse binary encoding and a WDL target from a strong teacher; in another domain, the encoding might be dense and continuous, the loss might be a regression or a contrastive objective, and the target might come from human demonstrations, simulation, or self-play. The dispatcher’s input features would likewise be domain-specific: in chess they are the L1 activations, but in another model they could be any intermediate representation that is cheap to compute and informative about the learning signal. These choices affect performance but do not alter the underlying method. The central claim, that per-sample gradients provide a useful signal for partitioning a state space in a way that supports efficient mixture-of-experts inference, is independent of the domain in which it is tested.

Chapter 4

Implementation: NNUE and MoE for Chess

4.1 Dataset and Teacher

The quality of an NNUE evaluation function depends critically on the dataset used for training. For this work, we constructed a dataset of approximately 105 million chess positions extracted from human games and labeled with high-quality value estimates from a strong teacher network, **Leela Chess Zero** (Lc0), the *spiritual* successor of *AlphaZero*.

4.1.1 Dataset Construction

The raw positions are sourced from **Lichess** monthly PGN archives, containing standard-rated games played by humans across all time controls. We filter games to include only those with at least 16 moves, excluding very short games that often end in early blunders or resignations and would introduce noisy or uninformative positions into the training set. From each remaining game, we sample positions randomly with probability 5%, ensuring a diverse and representative collection of states across all phases of play, and mostly avoiding highly correlated positions.

The dataset retains the natural distribution of game phases found in human play: a majority of middlegame positions, with fewer openings and endgames. We intentionally avoid resampling to balance phases, as the natural distribution better reflects the positions the engine will encounter during actual play. Each position is stored as a FEN string along with the multiplicity of the position, a visit count that may be used for optional weighting during training.

A small fraction of the dataset is also a collection of interesting tactical positions, and high level bot games. They capture a portion of the state space that may be outside of regular human play.

4.1.2 Feature Encoding

For the NNUE model, each FEN string is encoded as a sparse binary feature vector of length $d_{\text{in}} = 844$. This encoding is designed to capture both the positional and tactical structure of the board in a form suitable for the accumulator layer.

The 844 features are divided into two categories. The first comprises 716 base features that encode piece-square pairs, representing the presence of each piece type on each square. The feature set is pruned to remove impossible pawn ranks and compressed to reduce redundancy. In the 844-dimensional SARDINE encoder, the king plane is compressed from 64 squares to 32, saving features. The remaining 128 tactical features encode dynamic aspects of the position, specifically which pieces are under attack and which pieces are attacking the king. These features provide the network with explicit information about immediate tactical threats.

A key design choice is the **dual-POV** encoding: for each position, the encoder produces two sets of sparse indices, one from the perspective of the **side-to-move** (STM) and one from the opponent’s perspective (obtained by flipping the board rank-wise and swapping colors). This dual representation allows the network to learn symmetric evaluations and is consistent with the NNUE architecture’s ability to evaluate positions from either player’s viewpoint.

The encoded features are pre-computed and stored in `.npz` slices for efficient loading during training.

4.1.3 Data Splits

The dataset is partitioned into training and test splits. The training set consists of approximately 105 million positions, distributed across 165 slices for balanced I/O and stochastic sampling. The test set comprises a random portion of 1% of the positions that are held out of the training set.

4.1.4 Teacher Model: Lc0

To provide accurate target labels, we use **Lc0** (Leela Chess Zero) as the teacher model. Lc0 is a *convolutional neural network* trained via self-play reinforcement learning, following the AlphaZero paradigm. Its value head outputs a probability distribution over the three possible game outcomes, Win, Draw, and Loss, from the perspective of the side-to-move:

$$p_{\text{WDL}}(s) = \text{softmax}(\text{logits}(s)) = (p_W, p_D, p_L).$$

From this distribution, we compute the scalar expected reward:

$$v(s) = p_W - p_L \in [-1, +1],$$

which represents the expected outcome of the game from the current position. This scalar is the training target for the NNUE value head.

Lc0 is chosen as the teacher for several reasons. First, it natively outputs WDL probabilities, which align directly with the NNUE’s output head. Second, its strength, rated well above 3500 Elo, makes it a highly reliable source of positional evaluations. Finally, Lc0 is open-source and provides pre-trained networks, making the labelling pipeline reproducible.

For this work, we label positions using Lc0’s latest best network (e.g., `791556.pb.gz` from the Lc0 training server). We run Lc0 in UCI mode with `-show-wdl` enabled and evaluate each position with a single MCTS search. While depth-1 evaluations may occasionally miss short-term tactics, the resulting label noise is acceptable given the target Elo range of the engine (approximately 1700). For a cleaner but more expensive relabelling, one could increase the search depth.

4.1.5 Labelling Pipeline

The complete labelling pipeline is straightforward. We parse Lichess PGNs and sample positions uniformly at random from each game, saving FEN strings and visit counts. This reduces the correlation between the positions in the final dataset. For each unique FEN, we invoke Lc0 in *UCI mode* at depth 1 to obtain WDL probabilities from the STM perspective. We then proceed to save the WDL probabilities alongside the FEN and visit counts in JSON format. In the encoding step we pre-compute the 844-dimensional sparse feature vectors (both STM and opponent POVs) and store them in `.npz` slices for efficient training. The final result is a dataset of pairs of sparse input board positions and their relative WDL probabilities.

4.2 Base NNUE Architecture

The base NNUE (Efficiently Updatable Neural Network) model serves as the foundation upon which the Mixture of Experts extension is built. Its architecture is designed to balance representational capacity with the stringent memory and computational constraints of the target hardware. The model follows the dual-perspective paradigm introduced by the original NNUE design, but incorporates modifications tailored to the specific feature set and bucketing objectives of this work.

4.2.1 Model Overview

The base model f_θ is a feed-forward neural network with three parameterised layers: a shared sparse first layer (L1), a dense second layer (L2), and a linear output head, with *softmax* activations. The input to the model is the sparse binary feature representation described in Section 4.1.2, consisting of 844

active features per perspective. The model processes both the side-to-move (STM) and opponent perspectives through the same L1 layer, producing two accumulator vectors that are later concatenated and passed through the remaining layers.

Formally, the model computes:

$$h_{\text{own}} = W_{\text{L1}} x_{\text{own}}, \quad h_{\text{opp}} = W_{\text{L1}} x_{\text{opp}},$$

where $x_{\text{own}}, x_{\text{opp}} \in \{0, 1\}^{844}$ are the sparse feature vectors for the two perspectives, and $W_{\text{L1}} \in \mathbb{R}^{d_{\text{in}} \times W}$ is the shared weight matrix of the accumulator layer. The output of the L1 layer is a pair of vectors $h_{\text{own}}, h_{\text{opp}} \in \mathbb{R}^W$, where W is the hidden dimension of the accumulator, set to $W = 128$ in this work.

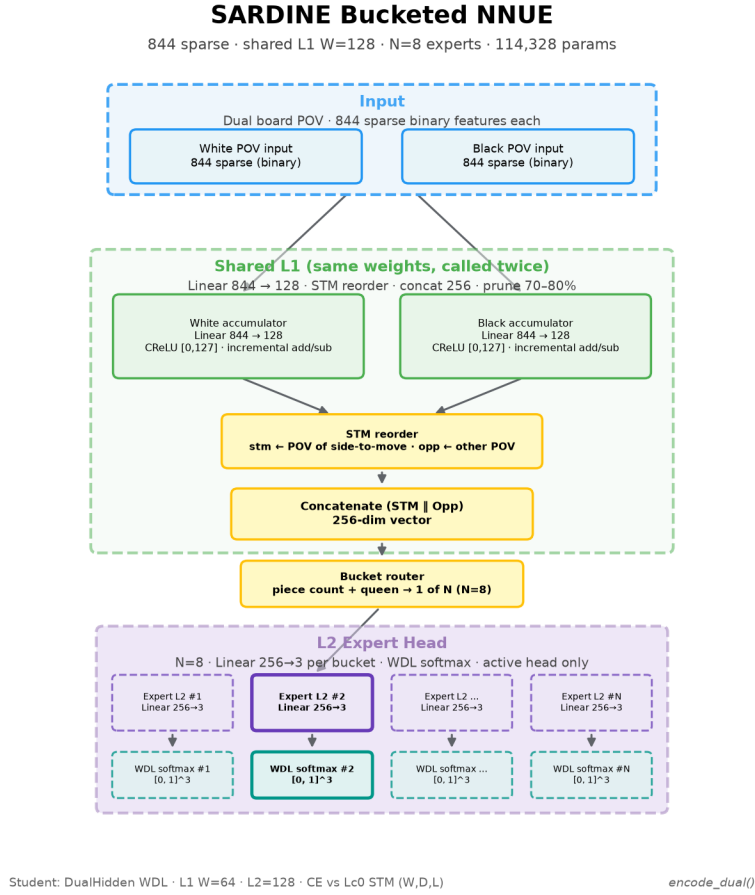


Figure 4.1: Dual-perspective NNUE architecture used as the base model. The shared L1 accumulator produces side-to-move and opponent activations, which are reordered, concatenated, and passed through the L2 layer and the three-way WDL output head.

4.2.2 Shared L1 Accumulator

The L1 layer, often referred to as the accumulator, is the defining component of the NNUE architecture. Its weight matrix W_{L1} is shared between the two perspectives, enabling the network to learn a common representation of board structure while retaining perspective-specific information through the different input features. The sparsity of the input features allows the accumulator to be updated efficiently: rather than recomputing the entire matrix-vector product for each new position, the network maintains the accumulator vector incrementally, adding or subtracting the contributions of features that change as pieces move.

The L1 activations are passed through a **CReLU** (Clipped Rectified Linear Unit) activation function, which maps each activation to the range $[0, 127]$:

$$a_{\text{own}} = \text{clamp}(h_{\text{own}}, 0, 127), \quad a_{\text{opp}} = \text{clamp}(h_{\text{opp}}, 0, 127).$$

This clipping is essential for integer quantization, as it bounds the dynamic range of the accumulator values and allows the use of low-precision integer arithmetic during inference.

4.2.3 Side-to-Move Reorder

Before concatenating the two accumulator vectors, we apply a *side-to-move (STM) reorder* to ensure that the expert head always receives the perspective of the current player first. The reordering is governed by the binary flag $\text{stm_white} \in \{0, 1\}$, which indicates whether White is to move:

$$h_{\text{first}} = \begin{cases} a_{\text{own}}, & \text{if } \text{stm_white} = 1, \\ a_{\text{opp}}, & \text{otherwise,} \end{cases}$$

$$h_{\text{second}} = \begin{cases} a_{\text{opp}}, & \text{if } \text{stm_white} = 1, \\ a_{\text{own}}, & \text{otherwise.} \end{cases}$$

The two vectors are then concatenated to form the input to the L2 layer:

$$h = [h_{\text{first}} \parallel h_{\text{second}}] \in \mathbb{R}^{2W}.$$

This reordering step is critical for making the evaluation perspective-invariant: the network always receives the board from the point of view of the side to move, and the output $v \in [-1, +1]$ is always interpreted as the expected reward for the current player, regardless of colour.

4.2.4 L2 Layer and Output Head

The concatenated vector h is passed through a dense L2 layer with hidden dimension $H = 256$:

$$z = \text{ReLU}(W_{L2} h + b_{L2}),$$

where $W_{L2} \in \mathbb{R}^{2W \times H}$ and $b_{L2} \in \mathbb{R}^H$ are the weight matrix and bias of the L2 layer. The ReLU activation introduces non-linearity and has been shown to work well with the sparse accumulator features.

Finally, the L2 activations are projected to a three-dimensional output representing the logits for Win, Draw, and Loss probabilities:

$$\text{logits} = W_{\text{out}} z + b_{\text{out}},$$

where $W_{\text{out}} \in \mathbb{R}^{H \times 3}$ and $b_{\text{out}} \in \mathbb{R}^3$. During training, these logits are converted to probabilities via the softmax function:

$$p_{\text{WDL}}(s) = \text{softmax}(\text{logits}(s)) = (p_W, p_D, p_L).$$

The scalar evaluation used for search is obtained as $v = p_W - p_L$, the expected reward from the side-to-move perspective.

4.2.5 Training Objective

The model is trained to minimise the **soft cross-entropy** between its predicted WDL distribution and the teacher labels provided by the Lc0 value function. Let $\mathcal{O} = \{W, D, L\}$ denote the set of possible game outcomes. For a batch of N positions with teacher probabilities $\hat{p}_i = (\hat{P}_i(o))_{o \in \mathcal{O}}$ and model outputs $p_i = (P_i(o))_{o \in \mathcal{O}}$, the loss is:

$$\mathcal{L} = -\frac{1}{N} \sum_{i=1}^N \sum_{o \in \mathcal{O}} \hat{P}_i(o) \log P_i(o).$$

The outer sum averages the loss over the batch, while the inner sum accumulates the contribution of each outcome for a given position.

This loss is well-suited to the task because the teacher labels are probability distributions rather than point estimates. Soft cross-entropy encourages the model to match the full outcome distribution, not merely its scalar expected reward, and therefore provides a richer training signal. The loss also handles the non-linearity introduced by the softmax in a principled way, and its gradient does not saturate at extreme values, unlike the squared error on the scalar evaluation $v = P(W) - P(L)$.

4.2.6 Parameter Count and Model Size

With $W = 128$ and $H = 256$, the total number of trainable parameters is approximately 175,000. This compact size is deliberately chosen to fit within the memory constraints of the target hardware: the L1 weights (844×128 `int8` values) dominate the parameter count, while the L2 and output layers contribute only a small fraction. The model is therefore both computationally efficient and storage-friendly, with a footprint that can be further reduced through pruning and quantisation.

4.2.7 Training Protocol

The base model is trained on the full training set (approximately 105 million positions) using the Adam optimiser with a learning rate of 10^{-2} , linearly decayed to 10^{-3} over the course of training. We use a batch size of 10,000 and train for up to 1000 epochs. The model’s performance is evaluated on a held-out test set of random positions, 1% of the total dataset, ensuring that generalisation is measured on unseen data. The training is conducted on a *DGX Nvidia Spark GPU*.

4.3 Sample Gradient Computation

4.3.1 Implementation with PyTorch

The sample gradients are computed using PyTorch’s automatic differentiation engine. For each batch of positions, a forward pass through the base model is performed to obtain WDL logits. Then, the soft cross-entropy loss is computed against the teacher labels. Finally, `torch.autograd.grad` is called to obtain the gradients of the loss with respect to the head parameters.

The computation proceeds as follows. For a batch of M positions, the base model $f_{w_{\text{base}}}$ produces logits $\ell_i \in \mathbb{R}^3$ for each position. The loss is computed as the average soft cross-entropy over the batch:

$$\mathcal{L}_{\text{batch}} = \frac{1}{M} \sum_{i=1}^M \mathcal{L}_{\text{CE}}(\text{softmax}(\ell_i), \hat{p}_i),$$

where $\hat{p}_i$ are the teacher’s WDL probabilities. We then call:

```
grads = torch.autograd.grad(
    loss,
    head_parameters,
    retain_graph=False,
    create_graph=False
)
```

The `head_parameters` list contains the trainable parameters of the L2 layer and the output head: $(W_{L2}, b_{L2}, W_{\text{out}}, b_{\text{out}})$. The L1 weights are excluded, as they remain frozen throughout the entire pipeline.

The resulting gradient tensors are detached from the computation graph to free memory, then flattened and concatenated into a single vector per position:

$$\Delta_i = \text{concat}[\text{vec}(\nabla_{W_{L2}} \mathcal{L}_i), \nabla_{b_{L2}} \mathcal{L}_i, \text{vec}(\nabla_{W_{\text{out}}} \mathcal{L}_i), \nabla_{b_{\text{out}}} \mathcal{L}_i].$$

The computation is parallelised across the GPU and performed in a single pass over the dataset. Gradients are computed in batches of 1024 positions, and the resulting vectors are accumulated on disk rather than held

in memory, preventing memory exhaustion. The `retain_graph=False` option ensures that the computational graph is freed after each batch, further reducing memory usage.

4.3.2 Parameter Selection

The gradients are computed with respect to the trainable parameters of the head only, namely the weights and biases of the L2 layer and of the output layer: $(W_{L2}, b_{L2}, W_{\text{out}}, b_{\text{out}})$. The L1 accumulator is deliberately excluded. This choice follows from the design of the method: the L1 representation is shared across all experts and frozen throughout the pipeline, so its parameters are never updated during expert fine-tuning and their gradients are irrelevant to the partition. Including them would also inflate the dimensionality of the gradient vectors by an order of magnitude, since the L1 weights alone account for the majority of the model’s parameters, and would make both storage and clustering substantially more expensive without contributing information that the partition can use.

The selected parameters are flattened and concatenated into a single vector per position. If the L2 layer has weight matrix $W_{L2} \in \mathbb{R}^{2W \times H}$ and bias $b_{L2} \in \mathbb{R}^H$, and the output layer has $W_{\text{out}} \in \mathbb{R}^{H \times 3}$ and bias $b_{\text{out}} \in \mathbb{R}^3$, the resulting gradient vector has dimension $P_{\text{head}} = 2W \cdot H + 4H + 3$, which for the architecture used in this work amounts to 66,563 values per position. This is the representation that is normalised, stored, and clustered in the subsequent steps.

4.3.3 Normalisation

Each gradient vector is L2-normalised before storage, so that the clustering operates on directions rather than magnitudes. The normalisation is applied element-wise to every vector in the batch:

$$\Delta_i^{\text{norm}} = \frac{\Delta_i}{\|\Delta_i\| + \epsilon},$$

where $\|\Delta_i\|$ is the Euclidean norm of the flattened gradient and $\epsilon = 10^{-8}$ prevents division by zero for degenerate vectors. The operation is vectorised across the batch: the norms are computed in a single reduction over the flattened buffer. In practice, no gradient in the dataset is exactly zero, but the epsilon guard is retained for safety.

The same normalisation is applied uniformly to all gradients. No per-dimension standardisation is performed, and no additional scaling is introduced. This ensures that the stored vectors all lie on the unit hypersphere, so that the Euclidean distance between two normalised vectors is a monotone function of their cosine similarity, and the clustering algorithm effectively operates on angles rather than magnitudes.

4.3.4 Storage and Memory Management

Computing and storing sample gradients for 105 million positions presents a significant practical challenge. Each gradient vector has dimension $P_{\text{head}} = 66,563$, corresponding to the flattened parameters of the L2 layer and output head. Storing the full set in 32-bit floating-point would require approximately 28 TB. We therefore adopt half-precision storage, reducing this to roughly 14 TB while preserving sufficient numerical precision for clustering, as the gradients are normalised and clustered based on their directions rather than their exact magnitudes.

The gradients are stored in memory-mapped `.npy` files, which provide efficient random access without loading the entire dataset into memory. Computation proceeds in batches of 1024 positions, with each batch written to disk immediately after computation and the memory-mapped array pre-allocated to avoid accumulating gradients in GPU or CPU memory. For exploratory analysis, gradient computation can be performed on a subset of the dataset, but for the final MoE architecture we use the full dataset.

4.3.5 Computational Cost and Timing

Computing sample gradients is the most expensive step of the pipeline after base training itself. Unlike inference, which requires only a forward pass through the model, gradient computation requires both a forward and a backward pass for every batch. The backward pass is the dominant cost, since it involves propagating gradients through the L2 layer and the output layer. The L1 accumulator is frozen and excluded from differentiation, so no gradient is propagated through it; this keeps the backward pass confined to the small head and avoids the cost of differentiating through the largest layer of the network.

The computation is *embarrassingly parallel* across batches. Each batch is independent of the others, and the only shared state is the model weights, which are read-only during this step. On a GPU, this parallelism is exploited by processing batches of 1024 positions at a time and running them through the model in sequence, with the backward pass overlapping the next forward pass through the use of asynchronous kernel launches. On a CPU, the same batching strategy applies, but the throughput is substantially lower, and the per-position cost becomes the bottleneck. For this reason, the gradient computation is performed on a GPU, where the throughput is roughly two orders of magnitude higher than on a modern multi-core CPU. The exact timing depends on the batch size, the size of the head, and the speed of the storage device used to write the gradients, since the I/O can become the limiting factor when the model itself is small.

The batching strategy is designed to balance memory usage and throughput. A batch size of 1024 positions is large enough to saturate the GPU and

small enough to fit comfortably in device memory, including the activations needed for the backward pass. Each batch is processed independently, and the resulting gradient vectors are written directly to the memory-mapped output file before the next batch begins. This has two effects: it bounds the memory footprint of the step to a single batch, regardless of the total dataset size, and it makes the computation resilient to interruption, since the gradients already written to disk remain valid. The main cost of this approach is that the GPU is idle during the write phase. For large batches, the write time is negligible compared to the backward pass; for small batches, the overhead becomes more significant, which is one of the reasons for choosing a batch size at the upper end of what the memory allows.

Under the configuration used in this work, computing gradients for the full dataset of 105 million positions is performed on a DGX Nvidia Spark GPU, with the storage writes contributing a minor fraction of the total time. The throughput scales roughly linearly with the number of positions, so the computation can be halted at any point and resumed later without loss, which is useful when a subset is sufficient for an exploratory analysis. A random subsample of the dataset is often used for tuning the clustering parameters before committing to a full pass, since the clustering cost also grows with the size of the stored gradients and the number of iterations.

4.4 Clustering

The clustering step operates on the full set of normalised sample gradients produced in the previous stage. The gradients are loaded from the memory-mapped file, and the clustering is performed with **Mini-Batch K-Means**, as described in Section 3.5.2. The choice of Mini-Batch K-Means over the standard K-Means follows from the size of the dataset: with millions of gradient vectors in dimension $P_{\text{head}} = 66,563$, the batch updates provided by Mini-Batch K-Means reduce both memory consumption and convergence time, while producing partitions that are nearly indistinguishable from those of the full algorithm on this scale.

The clustering is run for several values of B (2, 4, 8, 16, and 32), so that the effect of the number of experts on the final evaluation quality can be assessed. The Mini-Batch K-Means implementation uses a batch size of 10,000 gradients and is initialised with the k -means++ scheme, which improves the stability of the final partition by spreading the initial centroids across the data. The algorithm is run for a fixed number of iterations with early stopping on the within-cluster sum of squares, and the random seed is fixed so that the partitions are reproducible across runs. The trained centroids and the per-position cluster assignments are stored for the subsequent dispatcher training and expert fine-tuning steps.

As a secondary reference, Density Peak Clustering is applied to a random

subsample of the gradients, typically a few hundred thousand positions, to avoid the quadratic cost of computing pairwise distances on the full dataset. The number of clusters identified by this method is not used directly to set B , but serves as a qualitative check on whether the values explored with Mini-Batch K-Means are consistent with the natural structure of the gradient space. If the density-based algorithm consistently selects a number of clusters within the explored range, this provides some evidence that the partition is not being forced by an arbitrary choice of B .

The structure of the partition is visualised using two complementary tools. *Principal Component Analysis* is applied to a subsample of the gradients to project them onto the first few principal components, and the projection is coloured by cluster assignment to reveal the gross geometry of the partition. *t-SNE* is used on a smaller subsample to produce a non-linear embedding that preserves local neighbourhoods, which often reveals structure that the linear projection misses. Both visualisations are produced for several values of B , so that the progressive refinement of the partition can be inspected. These plots are not used to select the final value of B , which is decided on the basis of the downstream evaluation metrics, but they provide a useful qualitative check on the clustering and help identify whether any bucket is dominated by a narrow region of the state space.

4.5 Dispatcher Training

The dispatcher is trained to predict the cluster assignment of a position from the L1 activations produced by the frozen base model. The training data consists of the same set of positions used for clustering, with the cluster labels obtained from Mini-Batch K-Means serving as targets. Since the L1 weights are frozen and the activations depend only on the input features, the L1 activations can be computed once for the entire dataset and cached. This reduces the dispatcher training to a simple classification problem on a fixed set of features, and the entire procedure completes in a fraction of the time required for gradient computation or expert fine-tuning.

The input to the dispatcher is the concatenated L1 activations after the side-to-move reorder, $h = [h_{\text{first}} || h_{\text{second}}] \in \mathbb{R}^{2W}$, matching the input received by the L2 layer of the expert heads. Using the same representation as the heads ensures that the dispatcher sees the position in the same perspective-aligned form, so that the routing decision is consistent with the expert that will eventually process the position. An alternative would be to use only the side-to-move activations h_{own} , halving the input dimension and the number of dispatcher parameters. This was considered but not adopted, because the opponent’s perspective provides additional information about the position at negligible cost, and because the full concatenation matches the input of the expert heads more closely.

The dispatcher is a single linear layer mapping the $2W$ -dimensional input to B logits, with no hidden layers and no non-linear activation beyond the softmax used during training. The softmax is discarded at inference, where the predicted bucket is simply the argmax of the logits. This architecture was chosen for its minimal inference cost and because its limited expressiveness is less likely to overfit, as discussed in Section 3.6.2, and it is trained separately for each value of B explored in the clustering stage.

Training uses the standard cross-entropy loss between the predicted distribution over buckets and the one-hot cluster assignment. The optimisation is performed with Adam, using a learning rate of 10^{-2} decayed to 10^{-3} , a batch size of 1024, and early stopping on a held-out validation set comprising 10% of the training positions. Convergence typically occurs within a few epochs, and the validation accuracy is monitored to detect overfitting. No class weighting is applied, even when the cluster sizes are imbalanced, because the dispatcher’s objective is to approximate the partition as faithfully as possible, and artificially balancing the classes would distort the routing behaviour relative to the clustering. The trained dispatcher parameters are stored alongside the expert heads and loaded at inference time as part of the MoE model.

4.6 Expert Fine-Tuning

With the partition fixed and the dispatcher trained, the final training step produces the expert heads. Each expert is initialised from the base head and fine-tuned on the positions assigned to its bucket, with the L1 accumulator kept frozen throughout. The implementation follows the procedure described in Section 3.7, and the practical details of the training schedule are described here.

The fine-tuning runs over the buckets in sequence, each expert trained independently of the others. Because the L1 activations are pre-computed and cached, the fine-tuning only requires forward and backward passes through the small head, not the full model. This makes the procedure fast relative to base training, and it allows the experts to be trained with the same optimiser and loss used for the base model. The initialisation from the base head ensures that every expert starts from a well-trained configuration and that the differences between experts reflect the data they see, not differences in initialisation.

The choice of training schedule is guided by the size of the buckets and the risk of overfitting. In the present work the dataset is large enough that no bucket is small, and the training proceeds for a fixed number of sweeps over the bucket, with the cross-entropy on a held-out portion of the bucket serving as the stopping signal. The held-out portion is drawn from the same bucket, so the stopping criterion reflects performance on the type of positions

the expert is intended to handle. When a bucket is small, early stopping is essential, because an expert trained for too long on a limited set of positions will memorise them rather than generalise. When a bucket is large, the risk is lower, and the training can proceed for more sweeps without overfitting, but the gains from additional sweeps diminish quickly and the additional cost is not justified. The schedule used in the experiments is one to two sweeps for each bucket, with a learning rate lower than the one used for base training to avoid destabilising the shared representation.

The results of the fine-tuning are evaluated per bucket, using the cross-entropy of the expert on its own held-out data. This is the same metric used to monitor the base model, and it allows a direct comparison between the expert and the base head on the subset of positions that the expert was trained for. The per-bucket cross-entropy is reported in Chapter 5, along with the corresponding metrics for the base model and the dispatcher accuracy, so that the effect of specialisation can be assessed separately from the effect of routing.

4.7 Final MoE Model

The final MoE model assembles the three components produced by the pipeline into a single evaluation function: the frozen L1 accumulator, the trained dispatcher, and the B expert heads. In implementation terms, these correspond to three sets of parameters loaded from disk at startup and held in memory for the duration of the search. The L1 weights are shared and read-only, the dispatcher is a single linear layer with $2W \times B$ weights and B biases, and each expert head contains its own L2 and output parameters initialised from the base head and fine-tuned on its bucket.

The inference path is linear and adds only one stage to the base model. The engine encodes the position into its sparse feature representation and updates the L1 accumulator incrementally, as it does for the single-head model. The resulting activation vector, after the side-to-move reorder, is passed to the dispatcher, which computes B logits and selects the index of the maximum. The corresponding expert head is then evaluated on the same activation vector, producing WDL logits that are converted to probabilities via the softmax, and the scalar evaluation is derived as $v = p_W - p_L$ from the side-to-move perspective. No other expert is evaluated, and no ensembling is performed. The routing decision is therefore a single matrix-vector multiplication followed by an argmax, and the cost of the head forward pass is identical to that of the base model.

At the implementation level, the expert heads are stored as separate parameter blocks but share the same architecture, which allows the forward pass to be dispatched to the appropriate block using a single index. The dispatcher parameters and the expert heads are stored separately from the

L1 weights, so that the shared representation can be loaded once and reused regardless of the number of experts. The total memory footprint of the model is the sum of the L1 weights, the dispatcher weights, and the B expert heads, which for small B remains modest relative to the L1 layer. The additional cost introduced by the MoE architecture over the base model is therefore bounded by the size of the dispatcher and the $B - 1$ additional heads, and the inference path adds only the dispatcher operation to the cost of a single-head evaluation.

The assembled model is integrated into Cfish in the following sections. Its behaviour depends on the choices made throughout the pipeline, and in particular on the value of B and on the quality of the partition. The evaluation of the resulting engine, both in terms of the accuracy of the value estimates and in terms of the cost of the additional dispatcher, is reported in Chapter 5.

4.8 Cfish as the Host Engine

The Mixture-of-Experts NNUE developed in this chapter is not intended to run as a standalone evaluator. It is designed to replace the evaluation component of an existing chess engine, so that its behaviour can be assessed within the full search pipeline, and so that its cost can be measured against the constraints of the target hardware. The host engine selected for this purpose is Cfish, a port of Stockfish written in plain C by Ronald de Man and first published on GitHub in July 2016. Cfish shares the same search and evaluation logic as its C++ counterpart, but compiles to a smaller and more portable binary, which makes it a natural fit for embedded targets. Since August 2020, Cfish has included the NNUE evaluation ported from Stockfish, together with SIMD-optimised code paths for AVX2 and AVX-512.

The evaluation function in Cfish follows the *HalfKP* feature set, in which the active inputs are the positions of the non-king pieces relative to the square of the friendly king. The accumulator is updated incrementally as moves are made, and the network output is a scalar value in centipawns from the side-to-move perspective. The search is an alpha-beta variant with iterative deepening, move ordering, and a transposition table, and it calls the evaluation function at every leaf node. The integration described in the following section replaces the single NNUE head with the MoE architecture, while leaving the search tree, the move generation, and the transposition table unchanged.

The target hardware for the deployment is the Wio Terminal, an ARM board built around the ATSAMD51P19 Cortex-M4 processor running at 120 MHz. The board provides 512 KB of flash memory, 192 KB of RAM, and 4 MB of external flash. These figures impose hard limits on the size of the model and on the amount of state that can be kept in memory during

search. The L1 accumulator must fit within the available RAM alongside the transposition table and the search stack, and the expert heads and dispatcher must fit within the flash budget together with the engine code. The evaluation function must also be fast enough to sustain a reasonable number of nodes per second, which in turn determines the search depth the engine can reach within a given time control. The precise throughput depends on the quantisation strategy and on the efficiency of the integer arithmetic, and is measured as part of the experimental evaluation in Chapter 5.

4.9 Integration with Cfish

The integration of the NNUE MoE into Cfish follows the same structure that Cfish uses for the standard NNUE, with the addition of the dispatcher and the selection of the appropriate expert head. The entry point is the `evaluate()` function, which is called at every leaf node of the search. In the original engine, this function computes the accumulator from the current position, passes it through the single head, and returns a centipawn score. In the MoE version, the same accumulator is used, but the head is selected by the dispatcher before the output is computed.

The accumulator update is unchanged. Cfish maintains the accumulator incrementally as moves are made and unmade during the search, adding and subtracting the contributions of the pieces that change. Because the L1 weights are shared across all experts and frozen, the incremental update logic requires no modification: the accumulator is a single vector, computed once per position and reused by whichever expert the dispatcher selects. This is the practical consequence of freezing L1, and it is what makes the integration tractable. If each expert had its own accumulator, the engine would have to maintain B separate accumulator states and update them all on every move, which would multiply the cost of the most frequent operation in the search.

The dispatcher is invoked after the accumulator is computed and before the head forward pass. It reads the concatenated activations in the perspective-aligned order, computes B logits, and returns the index of the maximum. The forward pass of the selected expert is then performed on the same activation vector, using the same code path as the base head. The additional cost per evaluation is therefore one linear layer of size $2W \times B$ and one argmax over B values, both of which are inexpensive relative to the head forward pass. For the values of B explored in this work, the dispatcher adds a small but measurable increment to the per-node cost, and the effect on nodes per second is reported in the evaluation.

All weights are quantised to 8-bit integers for storage and inference. The L1 weights, the dispatcher weights, and the expert head weights are all stored as `int8`, and the biases are stored as `int32` to preserve the dynamic range of the accumulation. The accumulator itself is maintained in `int16`, which pro-

vides sufficient precision for the CReLU-clipped activations while keeping the vector within a compact representation. The multiply-accumulate operations at the dispatcher and the head use int32 accumulators, which is the standard integer path in NNUE implementations and is supported efficiently by the Cortex-M4 DSP instructions. The output of the head is a set of three int32 logits, converted to a scalar evaluation through a fixed-point approximation of the softmax difference $p_W - p_L$.

The memory layout is organised so that the largest component, the L1 weight matrix, is stored in the external flash and accessed through the board’s memory-mapped interface, while the accumulator and the search state reside in the internal RAM. The dispatcher and the expert heads are small enough to be stored in the internal flash alongside the engine code, which avoids the latency of external memory access on the critical path. The expert heads are stored as a contiguous block indexed by the dispatcher output, so that selecting an expert amounts to offsetting a pointer into the block rather than searching a data structure. This layout keeps the per-evaluation memory access pattern predictable and cache-friendly, which matters on a device with no hardware cache.

The integration is validated in two stages. First, the MoE model is run on a set of positions with known evaluations to confirm that the dispatcher selects the correct expert and that the output matches the reference implementation on the host. Second, the full engine is run on a set of test positions to measure nodes per second and search depth against the unmodified Cfish with the same base NNUE, so that the cost of the MoE extension can be isolated from the cost of the search itself. The results of these measurements are reported in Chapter 5, together with the evaluation accuracy of the resulting engine.

Chapter 5

Experiments and Results

5.1 Experimental Setup

5.2 Evaluation Metrics

5.3 Results

5.3.1 Linear Model

5.3.2 Single-layer FFNN

5.3.3 Double-layer FFNN

5.3.4 Base NNUE

We also tested the soft cross-entropy of the base NNUE model as a function of the size of the dataset, from 50k samples all the way to 105 million. The model has a 2×128 -neuron accumulator layer and a 256-neuron L2 layer, which lies on the larger side of the models we trained.

```
python3.12 -u scripts/train_nnue-gpu.py --epochs 100 --lr 0.01
--lr-end 0.001 --hidden-dim 128 --hidden2-dim 256 --test-
fraction 0.01 --batch-size 10000 --batches-per-epoch 512 --
train-val-subset-size 10000 --run-name
dual_h128_H256_e100_bpe512_bs10000 --plot plots/
dual_h128_H256_e100_bpe512_bs10000.png
```

We found that, below 7 million positions, the model clearly overfits the training set. On the other hand, above 7 million positions no overfitting is visible. These figures are to be kept in mind when partitioning a dataset for the MoE, as the per-expert data should never fall below this threshold.

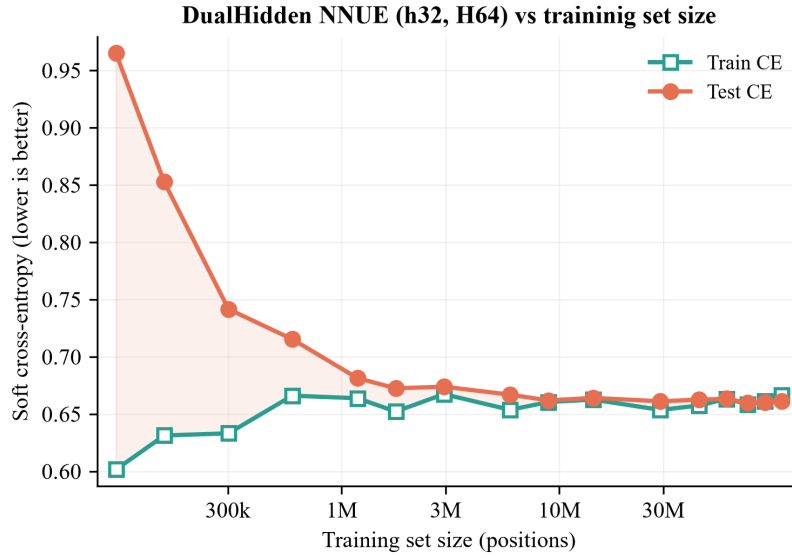


Figure 5.1: Soft cross-entropy of a smaller base NNUE as a function of training-set size.

5.3.5 MoE NNUE — bucketing by L1 — fixed B

5.3.6 MoE NNUE — bucketing by L1 — variable B

5.3.7 MoE NNUE — bucketing with sample-gradients — fixed B

5.3.8 MoE NNUE — bucketing with sample-gradients — variable B

5.4 Results: Comparison of the Models

5.5 Visualization: Clustering

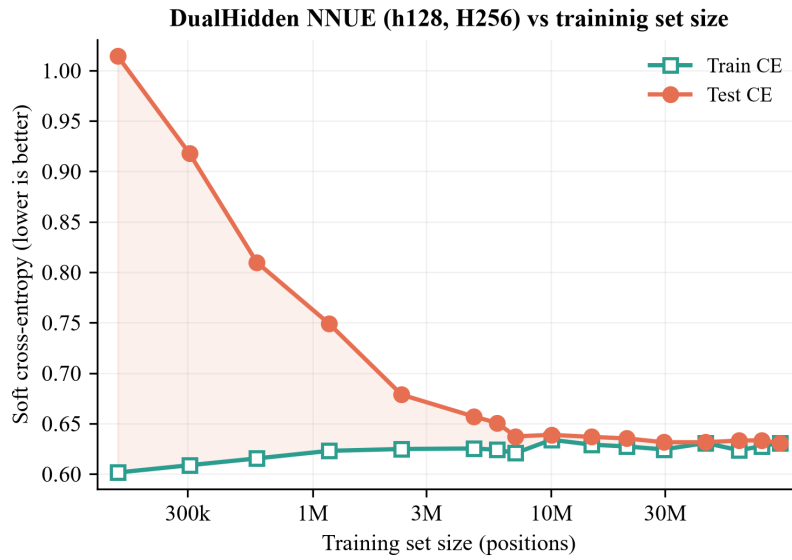


Figure 5.2: Soft cross-entropy of the $W = 128$, $H = 256$ base NNUE as a function of training-set size.

Chapter 6

Discussion

6.1 Interpretation of Results

6.2 Limitations

6.3 Generalisation

6.4 Comparison with Related Work

Chapter 7

Conclusion and Future Work

7.1 Summary of Contributions

7.2 Future Work

7.3 Closing Remarks

Appendix A

Supplementary Material

A.1 Extra tables and hyperparameters

A.2 Extra plots

A.3 Implementation notes

Bibliography

- [1] Yinghao Li et al. “Ensembles of Low-Rank Expert Adapters”. In: *International Conference on Learning Representations*. 2025. arXiv: [2502.00089](#).
- [2] Shrihari Sridharan et al. “GradientSpace: Unsupervised Data Clustering for Improved Instruction Tuning”. In: (2025). arXiv: [2512.06678](#).
- [3] David Silver et al. “A general reinforcement learning algorithm that masters chess, shogi, and Go through self-play”. In: *Science* 362.6419 (2018), pp. 1140–1144.
- [4] William Fedus, Barret Zoph, and Noam Shazeer. “Switch Transformers: Scaling to Trillion Parameter Models with Simple and Efficient Sparsity”. In: *Journal of Machine Learning Research* 23.120 (2022), pp. 1–39.
- [5] Nan Du et al. “GLaM: Efficient Scaling of Language Models with Mixture-of-Experts”. In: *Proceedings of the 39th International Conference on Machine Learning*. 2022, pp. 5547–5569.
- [6] Edward J. Hu et al. “LoRA: Low-Rank Adaptation of Large Language Models”. In: *International Conference on Learning Representations*. 2022.
- [7] Ryotaro Kawata et al. “Mixture of Experts Provably Detect and Learn the Latent Cluster Structure in Gradient-Based Learning”. In: *Proceedings of the 42nd International Conference on Machine Learning*. 2025. arXiv: [2506.01656](#).
- [8] Jipeng Zhang et al. “TAGCOS: Task-agnostic Gradient Clustered Core-set Selection for Instruction Tuning Data”. In: (2024). arXiv: [2407.15235](#).
- [9] Bach Ngo and Nguyen Hoang Khoi Do. “Hexaïssa: Standing on Giants’ Shoulders—Routing the Best Chess Engines with Mixture-of-Experts and Latent Reward Learning”. In: *Proceedings of the AAAI Conference on Artificial Intelligence*. Vol. 40. 29. 2026, pp. 24532–24540. DOI: [10.1609/aaai.v40i29.39636](#).

BIBLIOGRAPHY

- [10] Felix Helfenstein et al. “Checkmating One, by Using Many: Combining Mixture of Experts with MCTS to Improve in Chess”. In: (2024). arXiv: [2401.16852](#).
- [11] Geoffrey Hinton, Oriol Vinyals, and Jeff Dean. “Distilling the Knowledge in a Neural Network”. In: (2015). arXiv: [1503.02531](#).
- [12] J Rosser. “Gradient Atoms: Unsupervised Discovery, Attribution and Steering of Model Behaviors via Sparse Decomposition of Training Gradients”. In: (2026). arXiv: [2603.14665](#).
- [13] Alex Rodriguez and Alessandro Laio. “Clustering by fast search and find of density peaks”. In: *Science* 344.6191 (2014), pp. 1492–1496.
- [14] Peter J. Rousseeuw. “Silhouettes: A graphical aid to the interpretation and validation of cluster analysis”. In: *Journal of Computational and Applied Mathematics* 20 (1987), pp. 53–65.

Ei fu. Siccome immobile,
Dato il mortal sospiro,
Stette la spoglia immemore
Orba di tanto spiro

Alessandro Manzoni – *Il Cinque Maggio*